



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Defending Against Browser Fingerprinting: Web Privacy Preserving Chromium Extension

LICENSE THESIS

Graduate: **Ioana - Ștefania RÎSTEIU**

Supervisor: **Conf.Dr.Ing. Adrian Coleșa**

Co-Supervisor: **As.Ing. István-Attila Császár**

2026

Contents

Chapter 1	Introduction	1
1.1	Stateless Tracking	1
1.2	Fingerprinting Techniques and Defenses	1
1.3	Profile Pooling	2
1.4	Thesis Structure	3
Chapter 2	Project Objectives	4
2.1	Problem Statement	4
2.2	Primary Objective	4
2.3	Secondary Objective	4
2.3.1	Session Consistency	4
2.3.2	Session Independence	5
2.3.3	Real Profiles	5
2.3.4	Anti-Detection Measures	5
2.3.5	API Coverage	5
Chapter 3	Bibliographic Research	7
3.1	Browser Fingerprinting: Origins and Entropy	7
3.2	Large-Scale Measurement of Online Tracking	8
3.2.1	The OpenWPM Platform	8
3.2.2	The Tracking Ecosystem	8
3.2.3	Fingerprinting in the Wild	8
3.3	Randomization as a Defense Strategy	9
3.3.1	Exploiting Specification Flexibility	9
3.3.2	Perturbing Multimedia Renderings	9
3.3.3	Evaluation	10
3.4	Fingerprinting Effectiveness at Large Scale	10
3.4.1	Key Findings	10
3.4.2	Fragility of Non-Unique Fingerprints	11
3.4.3	Simulated Future Scenarios	11
3.4.4	Implications for Defense Design	11
3.5	Summary and Relation to the Present Project	12
Chapter 4	Analysis and Theoretical Foundation	13
4.1	Fingerprinting Vectors Covered	13
4.1.1	HTTP Request Headers	15
4.1.2	JavaScript API Vectors	16
4.2	Two-Layer Architecture	18
4.2.1	<i>Layer 1</i> : HTTP Headers (Network)	18
4.2.2	<i>Layer 2</i> : JavaScript APIs	18
4.2.3	Synchronization between Layers	19
4.3	Values Substitution Mechanism	19
4.3.1	Consistency vs. Diversity	19
4.3.2	Pseudo-Random Number Generator	20
4.3.3	Session Seed	20

4.3.4	Mulberry32 Hash-Based Generator	20
4.3.5	Separate PRNG Instances	21
4.4	Value Pool Strategy	21
4.4.1	Pool Construction	21
4.4.2	Weighting Through Repetition	22
4.4.3	Session Consistent Identity	23
4.5	Interception Strategies	23
4.5.1	Prototype Property Replacement	23
4.5.2	Proxy-Based Interception	24
4.5.3	toString() Override	24
Chapter 5	Detailed Design and Implementation	26
5.1	Extension Architecture	26
5.2	Profile Initialization and Synchronization Sequence	29
5.3	Function Masking	30
5.4	JavaScript API Hooking Implementation	31
5.4.1	Navigator	31
5.4.2	Screen and Window	33
5.4.3	Timezone	34
5.4.4	Canvas	37
5.4.5	WebGL	38
5.4.6	Audio	38
5.4.7	WebRTC	38
5.4.8	Speech Synthesis	39
5.5	Worker Context Propagation	39
5.6	Same-Origin Iframe Propagation	40
5.7	HTTP Header Rewriting	41
5.8	Design Decisions and Known Limitations	43
5.8.1	Design Decisions	43
5.8.2	Known Limitations	43
Chapter 6	Testing and Validation	44
6.1	Tools and Test Environment	44
6.2	Multi-User Fingerprint Test	44
6.2.1	Purpose and Structure	44
6.2.2	Verification	46
6.2.3	How It Is Run	46
6.2.4	Results	46
6.3	Audio Waveform Test	48
6.3.1	Purpose	48
6.3.2	How It Is Run	48
6.3.3	Results	48
6.4	External Validation	49
6.4.1	Cover Your Tracks	50
6.4.2	CreepJS	51

Chapter 7	User's Manual	53
7.1	System Requirements	53
7.1.1	Hardware	53
7.1.2	Software	53
7.2	Installation	53
7.3	Uninstallation	54
7.4	Usage	55
7.4.1	Everyday Use	55
7.4.2	What Changes	56
7.4.3	Verifying That the Extension Works	56
Chapter 8	Conclusions	57
8.1	Summary of Contributions	57
8.2	Analysis of Results	57
8.3	Possibilities for Improvement	58
Bibliography		59
Appendix A	Multi-Users Test Results	61
Appendix B	Published Papers	65

Chapter 1. Introduction

Online tracking is constantly evolving beyond traditional methods such as third-party cookies. A third-party cookie is a small piece of text created by a web server and stored on the user's device. They are primarily used by advertisers and analytics companies to monitor browsing behaviour and deliver personalised advertisements. Cookies provided a straightforward way of tracking users, as the tracking server only needed to read the identifier it had previously stored. However, in recent years, public awareness of privacy risks has grown, and regulatory measures such as the General Data Protection Regulation (GDPR) have led browser vendors to restrict or eliminate third-party cookies. Some browsers have blocked them entirely by default, while others have given users the option to delete or disable them.

1.1. Stateless Tracking

However, the restrictions imposed have not eliminated user tracking. Instead, they have accelerated the adoption of the stateless tracking techniques that do not require saving any data on the user's device. The most well-known technique is *browser fingerprinting*. Rather than placing an identifier on the device and reading it back later, fingerprinting constructs a unique identifier from the device itself: the combination of the browser version, operating system, screen resolution, installed fonts, graphics hardware, audio processing pipeline, timezone, and dozens of other properties that the browser exposes through standard JavaScript APIs and HTTP headers. Each property contributes a number of bits of entropy, and the combined fingerprint is often unique enough to distinguish a single user from millions of others [1].

Browser fingerprinting is difficult to block because it works passively and invisibly from the user's point of view. There are no alerts, permission prompts, or visible storage indicators. It uses standard web APIs that are part of everyday browsing activities such as loading pages, playing audio, and checking screen dimensions. The fingerprinting script appears as normal page code, and the collected data is sent using regular HTTP requests. Unlike cookies, users cannot inspect, modify, or delete their fingerprint because it is not stored locally. Instead, it is generated dynamically based on the device's attributes.

1.2. Fingerprinting Techniques and Defenses

Eckersley's 2010 research [1] showed that browser fingerprinting is highly feasible. By combining data such as the User-Agent string, screen resolution, system fonts, timezone and plugin list, it was possible to uniquely identify 83.6% of browsers in a dataset of over 470000 visitors. The findings indicate that each property adds entropy bits, and the combined entropy of multiple properties typically exceeds 33 bits, which is sufficient to distinguish a single user from the global internet population.

Since then, additional fingerprinting vectors have been discovered. Canvas fingerprinting, first documented by Englehardt and Narayanan [2], exploits the fact that different devices render the same drawing instructions with subtle differences at the pixel level caused by variations in GPU hardware, driver versions and anti-aliasing implementations. WebGL graphics hardware descriptors expose the GPU model and driver version. The Web Audio APIs reveal numerical properties of the Digital Signal Processing (DSP) pipeline. Even timezone and language settings contribute additional entropy to the overall fingerprint.

As fingerprinting techniques have grown more sophisticated, so have defences. These typically follow one of the two strategies.

The first, *blocking*, prevents access to fingerprinting APIs entirely. It is usually implemented by returning null values, throwing exceptions, or blocking network requests to known fingerprinting domains. Tools such as Privacy Badger [3] and uBlock Origin [4] employ variations of this approach. The problem with this technique is that it is detectable, the absence of an expected value is itself a signal. A browser that returns null for `navigator.hardwareConcurrency` or throws an exception on a `canvas.toDataURL()` call is immediately identifiable as running a privacy tool, which makes the user more unique rather than less [5].

The second strategy, *uniformity*, makes every user of a given tool present the same fixed fingerprint. Tor Browser [6] is the primary example: it reports identical screen dimensions, fonts, user-agent strings, and timezone across all installations, so that no individual user can be distinguished within the set. The limitation is that all Tor users share the same fingerprint, so while they cannot be told apart, they can still be identified as Tor users.

The third strategy, *noise injection*, returns different values each time a fingerprinting API is queried and slightly modified values. Brave Browser [7] applies per-session randomisation to canvas and audio outputs. FPRandom [5] randomises core browser objects to prevent stable fingerprinting. While noise injection avoids the blocking problem of being detectable, it introduces a consistency problem: a website that queries the same API twice within a single page load and receives two different answers can immediately detect that the values are being manipulated.

1.3. Profile Pooling

This thesis presents an alternative defence strategy that addresses the weaknesses of the approaches described above. Rather than blocking requests, presenting a uniform fixed identity, or randomising fingerprint values, the proposed approach replaces the user's real fingerprint with a consistent fake identity drawn from a pool of realistic profiles. This strategy, referred to as *profile pooling*, is built on three main properties:

1. **Session consistency:** Within a single browser session, every API call returns the same masked values. A website querying `navigator.userAgent` ten times in a row receives the same string each time. Canvas pixel data, audio waveforms, and timezone offsets all remain stable throughout the session. This is achieved through a seeded pseudo-random number generator (PRNG) that produces a deterministic sequence of choices throughout the session.

2. **Cross-session diversity:** When the user closes the tab and opens a new session, a fresh seed is generated, selecting a different profile from the pool. Two visits to the same website from the same device cannot be linked, because the fingerprint observed by the server is completely different.
3. **Profile credibility:** The spoofed values are not arbitrary constants or random noise. They are drawn from pools of values that reflect real-world browser identities, based on statistics from sources such as StatCounter [8], the Steam Hardware Survey [9], and the AmIUnique fingerprint dataset [10]. A user running the extension shows a fingerprint that seems normal and typical. It does not stand out as one of a privacy tool user.

The implementation takes the form of a Chromium browser extension built on the Manifest V3 platform. It is organised into two layers: HTTP header rewriting through the `declarativeNetRequest` API and JavaScript API interception through a content script injected before any page code executes. Together, these layers cover eight fingerprinting vectors: Navigator properties, Screen and Window dimensions, Timezone, Canvas, WebGL, Audio, WebRTC, and Web Workers.

1.4. Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter 2** defines the project objectives and scope.
- **Chapter 3** surveys the existing literature on browser fingerprinting techniques and defences.
- **Chapter 4** presents the theoretical foundation of the profile-pooling approach, including the threat model, the pseudo-random number generator design, and the entropy analysis of the value pools.
- **Chapter 5** describes the detailed design and implementation of each modified property.
- **Chapter 6** covers the testing and validation methodology, including results from a custom multi-user testing framework, the CreepJS fingerprinting analyser, and the EFF's Cover Your Tracks tool.
- **Chapter 7** provides a user manual for installing and using the extension.
- **Chapter 8** concludes with a summary of contributions, a critical analysis of the results, and directions for future work.

Chapter 2. Project Objectives

2.1. Problem Statement

As discussed in the previous chapter, browser fingerprinting enables websites to distinguish users by generating a unique identifier for each session. The existing defense strategies each have significant drawbacks. The blocking technique stops access to APIs, but the lack of a response indicates that a privacy tool is active, making it more detectable by tracking systems. Noise-based approaches provide different values for each API call, but if a website queries the same property twice, it will get two different results, which can be used for detection.

This project aims to create a browser extension that protects against fingerprinting by replacing the user's fingerprint with a consistent fake identity that is internally consistent, changes between sessions and looks like a real browser profile closely enough to prevent detection.

2.2. Primary Objective

The primary objective is to design and implement a Chromium browser extension that replaces the user's actual browser fingerprint with a plausible fake identity drawn from a pool of realistic profiles. The extension must work only within the Chrome Manifest V3 extension framework, without needing any changes to the browser or access to internal browser features beyond the extension API.

2.3. Secondary Objective

To meet the primary objective, the project is developed based on five important subobjectives that help achieve the project's purpose.

2.3.1. Session Consistency

Within a single browser session, all fingerprinting APIs should output consistent synthetic values, no matter how often they are accessed. For example, if a website queries `navigator.userAgent` ten times, it should get the identical string each time. Similarly, canvas colour perturbation should be applied using the same deterministic adjustment across all drawing operations, as this is part of the most basic anti-spoofing methods used by fingerprinting scripts.

To achieve this, the extension must use a deterministic process that generates an identical sequence of fake values for the duration of a session. This can be done by using a seeded pseudo-random number generator (PRNG), which creates a seed stored in the

sessionStorage. This storage persists across page navigations within the same tab but is cleared when the tab is closed.

2.3.2. Session Independence

Each new browser session should display a distinct fingerprint. When a user closes a tab and opens another, the extension must create a new seed and choose a different profile from the pool. This approach reduces the ability to link repeated visits, although the level of variation relies on how diverse the profile pool is. This directly undermines the purpose of fingerprinting, which is to recognise returning visitors.

2.3.3. Real Profiles

The modified values should be based on realistic browser profiles to make the fingerprint appear authentic. Using a Chrome version that was never released or a screen resolution that no actual monitor uses would reveal the fingerprint as fake.

The value pools must be constructed using publicly available statistics on browser versions, screen resolutions, GPU configurations, memory sizes and other hardware properties, drawn from datasets such as StatCounter [8], the Steam Hardware Survey [9], and IAmUnique [10].

2.3.4. Anti-Detection Measures

The extension must oppose common techniques used by fingerprinting scripts to detect whether browser APIs have been modified. These techniques include:

- Calling `toString()` on a browser function to check whether it returns the expected `[native code]` string or instead reveals the source code of a replacement function.
- Reading `name` property of functions to confirm it matches the expected native function.
- Comparing values from HTTP headers and JavaScript APIs to identify discrepancies between the two layers.
- Executing fingerprinting code within Web Workers or SharedWorkers, as they run in a separate execution context and do not inherit any modifications made to the main page.
- Creating same-origin iframes and reading browser properties from the fresh JavaScript domain inside them, bypassing hooks applied to the parent page.

2.3.5. API Coverage

The extension must cover the fingerprinting vectors most commonly used by the tracking scripts. Based on the research that will be discussed in the next chapter, the following nine vectors are in scope:

1. **Navigator properties** – User-Agent, User-Agent Data, platform, the number of CPU cores, the amount of device memory, and the user's preferred languages.
2. **Screen and Window** – screen resolution, available dimensions, colour depth, pixel ratio and outer window dimensions.

3. **Canvas** – the pixel output of the drawing surface, which is altered by applying small colour shifts to drawing operations, substituting the requested font with another font from a pool and introducing pixel-level changes to the exported image data. This approach is inspired by the techniques described in the FPRandom paper [5].
4. **WebGL** – GPU vendor and renderer strings on both WebGL1 and WebGL2 contexts.
5. **Audio** – offline audio processing pipeline output, with deterministic noise injected into a window of the audio buffer.
6. **Timezone** – timezone name, offset from UTC and all date-related functions that return or format local time, including the Date object’s string representations and its constructor.
7. **WebRTC** – the exposure of local and public IP addresses through the browser’s real-time communication interface. The connection policy is restricted so that only relay servers are used and no direct network information is revealed.
8. **Speech Synthesis** – the list of available text-to-speech voices reported by the browser, which is filtered to include only the voices that correspond to the spoofed language profile.
9. **Web Workers, SharedWorkers and iframes** – all of the above changed values are replicated inside Worker execution contexts by intercepting Worker creation and embedding the spoofed values directly into the Worker script before it runs. Same-origin iframes are also addressed by intercepting access to the iframe’s internal page.

Chapter 3. Bibliographic Research

This chapter reviews the research on browser fingerprinting and the defences proposed against it. The goal is to present the background on which this project — an anti-fingerprinting browser extension for Chromium — is based. The chapter is divided into four sections: the origins and measurement of browser fingerprinting, large-scale studies of online tracking, defences based on randomisation, and the effectiveness of fingerprinting when tested on larger populations.

3.1. Browser Fingerprinting: Origins and Entropy

The seminal study on browser fingerprinting was carried out by Eckersley [1], who deployed the Panopticlick website to collect device-specific attributes from 470,161 browsers between January and February 2010. The fingerprinting algorithm combined eight categories of information—User-Agent string, HTTP Accept headers, whether cookies were enabled, screen resolution, timezone, browser plugins and their versions, a list of system fonts obtained through Flash or Java applets, and partial supercookie support—into a single fingerprint for each visiting browser.

Eckersley used Shannon entropy (measured in bits) to quantify the identifying information carried by the fingerprint. The key finding was that the overall fingerprint distribution contained at least 18.1 bits of entropy, meaning that a randomly chosen browser would, on average, share its fingerprint with only one in roughly 286,777 others. Among browsers with Flash or Java enabled, 94.2% had unique fingerprints in the sample. Plugins and system fonts proved to be the most discriminating attributes, contributing 15.4 and 13.9 bits of entropy respectively.

An important part of the study concerned fingerprint stability over time. Among returning visitors, 37.4% exhibited at least one fingerprint change, driven by browser upgrades, plugin updates, or font installations. However, a simple heuristic algorithm could correctly link a changed fingerprint to its predecessor in 99.1% of the cases in which it ventured a guess, with a false positive rate under 1%. This demonstrated that fingerprints are not only highly unique but also trackable across configuration changes.

Eckersley also identified a key paradox: privacy-enhancing technologies can be self-defeating if they are not adopted widely enough. Tools such as Flash blockers or User-Agent spoofers made their users *more* distinctive, not less, because the privacy measure itself was a rare and recognizable configuration. The study noted that the Tor Browser was a commendable exception, as it had been designed with fingerprint resistance in mind from the outset. The recommended countermeasures included switching browser APIs from enumerable to confirm-only semantics (e.g., for fonts and plugins) and reducing the precision of version numbers exposed through JavaScript.

The principal limitation of the Panopticlick dataset is its bias toward technically educated, privacy-conscious users. As Eckersley acknowledged, the sample was not repre-

sentative of the global browser population, and the interactive nature of the site may have encouraged visitors to alter their configurations. Despite this, the study established the vocabulary and methodology (entropy, anonymity set size, surprisal) that all subsequent fingerprinting research has adopted.

3.2. Large-Scale Measurement of Online Tracking

Englehardt and Narayanan [2] conducted the largest web privacy measurement study reported up to that point, crawling the homepages of the Alexa top one million websites in January 2016. Their work had two major contributions: the OpenWPM measurement platform and a comprehensive census of stateful and stateless tracking techniques.

3.2.1. The OpenWPM Platform

OpenWPM [2] solved three engineering challenges that had limited previous measurement efforts: achieving scale through parallelism, providing comprehensive browser instrumentation, and enabling code re-use between studies. The platform drove a full-fledged Firefox browser via Selenium, rather than using a lightweight headless browser like PhantomJS. This choice was validated empirically: PhantomJS loaded roughly 30% fewer HTML resources and missed many advertising-related requests, making tracking measurements with a stripped-down browser unreliable. The instrumentation layer monitored HTTP requests and responses, JavaScript API calls (via custom getters and setters on prototype objects), and changes to on-disk storage, linking all observations back to the specific page visit that triggered them.

3.2.2. The Tracking Ecosystem

The census revealed that more than 81,000 distinct third-party domains were present on at least two first-party sites, but prevalence dropped off sharply: only 123 of those 81,000 appeared on more than 1% of sites. When grouped by parent organization, Google, Facebook, Twitter, and AdNexus were the only entities present on more than 10% of sites. The authors proposed a *prominence* metric, weighting third-party presence by the inverse rank of the host site, which produced rankings more robust to measurement scale than a simple site count.

Cookie syncing—the practice of third parties sharing user identifiers with each other—was found to be pervasive. Among the top 50 third parties involved in cookie syncing, the probability that a randomly chosen pair shared at least one cookie was 85%. Some cookies were synced to dozens of third parties, collectively covering over 60% of the top 1,000 sites.

3.2.3. Fingerprinting in the Wild

The study measured several fingerprinting techniques deployed on real websites. Canvas fingerprinting was found on approximately 1.6% of the top million sites, served by roughly 400 domains. A noteworthy trend was that the most prominent trackers from a 2014 study had largely stopped using canvas fingerprinting, suggesting that public scrutiny was effective against major players, while the total number of domains employing

the technique had grown, indicating that knowledge of it was spreading among smaller trackers.

The authors also reported, for the first time at scale, canvas-based font fingerprinting (present on roughly 3,250 sites), WebRTC-based local IP discovery (on 715 sites, 88.7% for tracking purposes), and AudioContext fingerprinting. The AudioContext technique works by processing an audio signal through a chain of Web Audio API nodes (oscillator, compressor, analyser) and hashing the resulting output buffer; differences in hardware and software audio stacks produce device-specific results. This technique was found on 512 sites via a single dominant script.

Critically, the study showed that existing privacy tools such as Disconnect, EasyList, and EasyPrivacy detected the majority of *prominent* fingerprinting scripts but missed the long tail: while 90% of scripts with a prominence above 0.01 were blocked, only 35% of those with a prominence above 0.0001 were. This finding underscored the need for complementary defense strategies beyond manually curated block lists.

The main limitations of this study are that only homepages were crawled (no internal page navigation), and the measurements were conducted from a single geographic vantage point (an AWS data center in the US East region).

3.3. Randomization as a Defense Strategy

Laperdrix, Baudry, and Mishra [5] proposed FPRandom, a modified version of Firefox that introduces controlled randomness into browser subroutines to break the stability of fingerprints across browsing sessions. While earlier defenses such as Blink [11] relied on heavyweight virtual-machine reconfiguration and PriVaricator [12] on policy-driven value substitution, FPRandom exploited two lighter-weight sources of flexibility inherent to browsers.

3.3.1. Exploiting Specification Flexibility

The first source is the gap between specification and implementation. The ECMAScript standard deliberately leaves certain behaviors unspecified—most notably the enumeration order of object properties (Section 13.7.5.15 of ES2016). In practice, every major engine chose a deterministic order, and Nikiforakis et al. [13] showed that this order is consistently different across browser families, versions, and even operating-system deployments, making it a fingerprinting vector capable of unmasking a browser that spoofs its User-Agent. FPRandom re-randomizes this order at each browser session by assigning random outcomes to the Latin-1 character comparisons used internally by Firefox’s `SortComparatorIds` structure, thereby preventing linkability while remaining compliant with the specification.

3.3.2. Perturbing Multimedia Renderings

The second source is perceptual tolerance in multimedia output. For canvas fingerprinting, FPRandom modifies the `ParseColor` function of the `CanvasRenderingContext2D` class so that each RGB channel is perturbed by a small delta per session. Unlike so-called “canvas poisoners” that add independent random noise per pixel (which can be averaged out by repeated tests, as noted by Mowery and Shacham [14]), FPRandom applies a consistent color shift across surfaces, making it resistant to repeated-measurement

attacks within a session. Additionally, the `SetFont` function substitutes the requested font with a randomly chosen system font, further altering the canvas hash.

For `AudioContext` fingerprinting, the authors introduced noise directly into the `AudioBuffer` processing routines by slightly decreasing the volume of processed buffers by a factor between 0.000 and 0.001. The resulting waveform differences, when zoomed to 1000%, were visible only as negligible jitter in the least significant decimal digits, while the hash produced by a fingerprinting script changed with every session.

3.3.3. Evaluation

FPRandom was evaluated against two real-world fingerprinting systems. Against the open-source `fingerprintjs2` library, 100 executions on FPRandom in Random mode produced 100 distinct hashes, while a vanilla Firefox produced the same hash every time. Against the Maxmind device tracking add-on, FPRandom returned different canvas hashes and different enumeration orders at each session.

Performance benchmarks showed negligible overhead: canvas rendering took approximately 0.18 ms versus 0.12 ms on vanilla Firefox; the JetStream JavaScript benchmark scores were statistically indistinguishable; and a crawl of the Alexa top 1,000 sites showed comparable page load times, with only a single site encountering a script error due to the enumeration-order modification. A user study with 20 participants found that half noticed subtle color differences in canvas renderings, 25% detected any difference in audio output, and those who did described it as “tiny” or “small.”

The main limitation is that FPRandom does not address static attributes such as the User-Agent string or screen resolution, and it requires modifying the browser source code rather than working as a standalone extension. The authors also note that a sufficiently motivated adversary could attempt statistical analysis to detect or remove the introduced noise, although no such behavior had been observed from real-world fingerprinting actors at the time.

3.4. Fingerprinting Effectiveness at Large Scale

Gómez-Boix, Laperdrix, and Baudry [15] collected 2,067,942 browser fingerprints over six months from one of the top 15 French websites, providing the first large-scale fingerprinting study on a general-audience commercial site rather than a privacy-focused test page.

3.4.1. Key Findings

The most striking result was that only 33.6% of fingerprints were unique in the dataset—far below the 83.6% reported by Panoptlick [1] and the 89.4% reported by AmIUnique [16]. For mobile devices, the figure dropped to 18.5%, compared with 81% from the AmIUnique study. This gap is primarily explained by the difference in audience: the commercial site attracted a broad, general population whose OS and browser distributions closely matched global StatCounter statistics, whereas Panoptlick and AmIUnique drew disproportionately many Linux users and privacy enthusiasts with unusual configurations.

The study confirmed that the most discriminating attributes differ by device class. On personal computers, the list of plugins remained the most distinctive attribute despite

its declining entropy (normalized entropy of 0.494 versus 0.817 in the Panoptick era), followed by canvas and fonts. On mobile devices, plugins contributed almost no entropy (0.011); instead, the User-Agent was the strongest signal (0.487), because manufacturers embed model numbers and firmware versions in it.

3.4.2. Fragility of Non-Unique Fingerprints

An important experiment tested whether non-unique fingerprints become unique when a single attribute changes. On personal computers, changing the content language, list of plugins, available fonts, or timezone caused over 85% of previously non-unique fingerprints to become unique. On mobile devices, language and timezone changes had a similar effect (over 60% became unique), while font and plugin changes had little impact due to the constrained software environments of smartphones. This fragility implies that even users who currently share a fingerprint with others can be individuated by any configuration change—a browser update, a language addition, or a timezone shift during travel.

3.4.3. Simulated Future Scenarios

Following the approach of Laperdrix et al. [16], the authors simulated the impact of foreseeable technical evolutions:

- **End of browser plugins.** Removing plugin diversity on personal computers reduced uniqueness from 35.7% to 16.5%, closely matching the mobile rate.
- **Standardized HTTP headers.** Reducing the identifying information in User-Agent and HTTP headers produced only a moderate decrease (to 31% on PCs, 16.2% on mobile).
- **Disabling JavaScript.** Removing all JavaScript-collected attributes dropped uniqueness to 0.7% on PCs and 4.3% on mobile, but at an unacceptable cost to web functionality.

These simulations suggested that the ongoing deprecation of NPAPI plugins and Flash—which has since been completed by all major browsers—would substantially narrow the fingerprinting surface for desktop machines, bringing their identifiability closer to that of mobile devices.

3.4.4. Implications for Defense Design

The study’s results have direct implications for the design of anti-fingerprinting tools. First, because audience demographics and geographic concentration strongly affect fingerprinting effectiveness, a defense that blends a user into a large and realistic anonymity set (e.g., common Windows/Chrome configurations in a given region) may be more effective than one that generates exotic or inconsistent profiles. Second, the declining importance of plugin-based attributes and the rising importance of canvas, AudioContext, and WebGL fingerprinting confirm that modern defenses must target these newer APIs. Third, the fragility of non-unique fingerprints reinforces the value of the session-based randomization strategy proposed by FPRandom [5]: even a small, imperceptible perturbation of a dynamic attribute can prevent an adversary from linking successive visits.

3.5. Summary and Relation to the Present Project

Table 3.1 summarizes the four principal studies reviewed in this chapter.

Table 3.1: Summary of the principal studies reviewed.

Study	Year	Dataset / Scale	Main contribution
Eckersley [1]	2010	470K fingerprints (Panopticlick)	Quantified fingerprint entropy; 83.6% unique
Englehardt & Narayanan [2]	2016	Top 1M sites (OpenWPM)	Measured fingerprinting in the wild; discovered Audio-Context FP
Laperdrix et al. [5]	2017	Modified Firefox (FPRandom)	Session-based randomization of canvas, audio, JS property order
Gómez-Boix et al. [15]	2018	2M fingerprints (commercial site)	Only 33.6% unique on a general audience

The present project—an anti-fingerprinting extension for Chromium-based browsers—draws on all four bodies of work. From Eckersley’s study, we adopt the principle that effective defense requires blending into a sufficiently large anonymity set, which our extension achieves by selecting profiles from pools of common Chrome-on-Windows configurations weighted by real-world market-share data. From Englehardt and Narayanan’s census, we identify the fingerprinting vectors that are actively deployed on the web (canvas, AudioContext, WebGL, WebRTC) and prioritize them in our spoofing strategy. From FPRandom, we borrow the core techniques of per-session color perturbation and audio-buffer noise injection, adapting them from a browser-source-code patch to a Manifest V3 content script that overrides JavaScript prototype methods via `Object.defineProperty` and `Proxy`. Finally, from Gómez-Boix et al.’s large-scale analysis, we take the lesson that realistic, regionally plausible profiles produce better privacy outcomes than exotic ones, guiding our choice of User-Agent strings, GPU renderer strings, language lists, and timezone values drawn from European market-share distributions.

Chapter 4. Analysis and Theoretical Foundation

The previous chapters introduced browser fingerprinting as a tracking technique, highlighted the weaknesses of existing defenses (blocking and noise injection), and defined the project’s objectives around the profile-pooling strategy. This chapter focuses on how the objective can be realized, presenting technical principles, algorithms used, and design decisions that support the extension’s implementation. Each section covers a specific part of the system, explaining both its operation and the reasons for choosing this approach over others.

4.1. Fingerprinting Vectors Covered

Before describing the architecture, Table 4.1 presents the specific fingerprinting vectors that the extension targets, along with some browser APIs that reveal each vector and the type of information each property exposes.

Table 4.1: Fingerprinting vectors addressed by the extension

Vector	API surface	Information leaked
User-Agent	<code>navigator.userAgent</code> , HTTP headers	Browser name, version, operating system
Client Hints	<code>navigator.userAgentData</code>	CPU architecture, platform version
Languages	<code>navigator.languages</code> , <code>accept-language</code>	User locale and language preferences
Hardware	<code>navigator.deviceMemory</code> , <code>navigator.hardwareConcurrency</code>	RAM size, CPU core count
Screen	<code>screen.width/height/colorDepth</code> , <code>window.devicePixelRatio</code> , <code>outerWidth</code> , <code>outerHeight</code>	Monitor resolution, colour depth, pixel density
Timezone	<code>Date.getTimezoneOffset()</code> , <code>Intl</code>	Geographic location
Canvas	<code>toDataURL()</code> , <code>fillStyle</code> , <code>strokeStyle</code>	Font rendering, GPU pipeline

Table 4.1 *Fingerprinting vectors addressed by the extension*

Vector	API surface	Information leaked
WebGL	<code>WEBGL_debug_renderer_info</code>	GPU vendor and model
Audio	<code>AudioBuffer.getChannelData()</code>	Floating-point audio stack behaviour
WebRTC	ICE candidates	Local and public IP addresses
Speech	<code>speechSynthesis.getVoices()</code>	Installed language packs

Figure 4.1 illustrates the browser fingerprinting process. A script embedded in a website queries various browser APIs, combines the collected data into a hash and transmits it to a tracking server. If the server has encountered the same hash previously, it recognises the user otherwise, it stores the hash for future recognition.

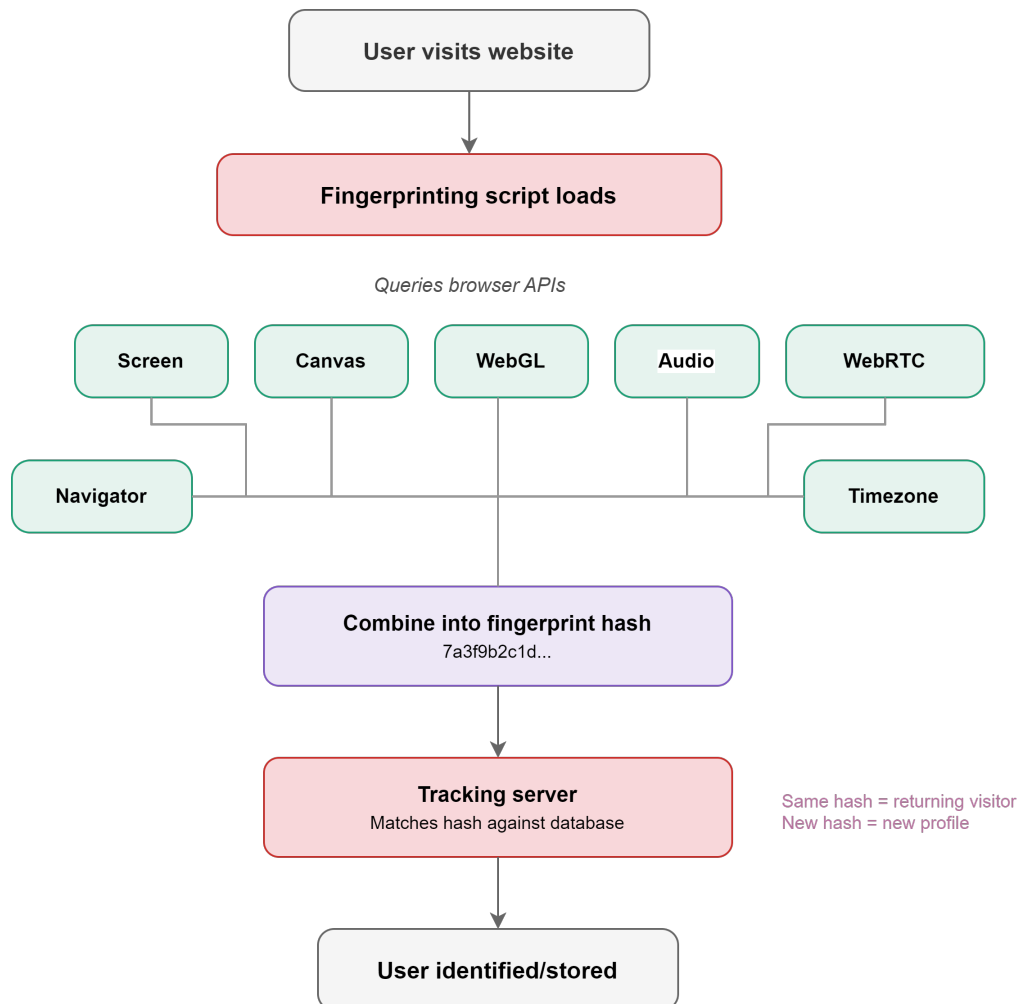


Figure 4.1: Browser fingerprinting process

4.1.1. HTTP Request Headers

Every time a browser loads a page, image, script, or any other resource, it sends an HTTP request to the server. This request consists of a **request line** (e.g., `GET /index.html HTTP/1.1`) followed by a block of **headers**. Headers are sent *before* any JavaScript runs on the page.

A simplified HTTP request from a real Chrome browser looks like this:

Listing 4.1: Example of HTTP request headers

```

1 GET /page.html HTTP/1.1
2 Host: www.google.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
4   AppleWebKit/537.36 (KHTML, like Gecko)
5   Chrome/147.0.0.0 Safari/537.36
6   OPR/131.0.0.0
7 Accept-Language: en-GB,en-US;q=0.9,en;q=0.8,
8   ro;q=0.7,id;q=0.6,de;q=0.5
9 Sec-CH-UA: "Opera";v="131", "Not.A/Brand";v="8",
10  "Chromium";v="147"
11 Sec-CH-UA-Mobile: ?0
12 Sec-CH-UA-Platform: "Windows"
```

User-Agent

The **User-Agent** header is the oldest and most widely used identification header. It is a single string that encodes the browser name, version number, rendering engine, and operating system.

Sec-CH-UA (Client Hints)

Sec-CH-UA is a newer header introduced as part of the **User-Agent Client Hints** specification. It was designed to replace the **User-Agent** string with a more structured format that is easier for servers to parse. The value is a comma-separated list of brand/version pairs: `"Opera";v="131", "Not.A/Brand";v="8", "Chromium";v="147"`.

Sec-CH-UA-Mobile

This header indicates whether the browser is running on a mobile device. The value is either `?1` (mobile) or `?0` (desktop).

Sec-CH-UA-Platform

This header reports the operating system name. Typical values are `"Windows"`, `"macOS"`, `"Linux"`, `"Android"`, or `"Chrome OS"`.

Accept-Language

The **Accept-Language** header tells the server which human languages the user prefers, in order of priority. Each language tag can include a quality weight (`q=0.0` to `q=1.0`) indicating relative preference: `en-GB,en-US;q=0.9,en;q=0.8,ro;q=0.7`.

4.1.2. JavaScript API Vectors

While HTTP headers are sent before any page code executes, the browser also exposes a set of JavaScript APIs that reveal device characteristics from within the page itself. These APIs were designed for different purposes, but each one leaks information that, when combined with others, can uniquely identify a browser instance.

User-Agent

The `navigator.userAgent` property returns the same identification string that the browser sends in the **User-Agent** HTTP header (4.1.1). It encodes the browser name, version, rendering engine and operating system in a single string.

On the JavaScript side, this property exists because web pages need to detect the browser before using engine-specific features.

Client-Hints

The `navigator.userAgentData` object is the JavaScript equivalent of the User-Agent HTTP header, but structured and more detailed. It exposes two levels of information: a basic list of browser brand names available immediately and a second set of more detailed values that must be explicitly requested, including the processor architecture, whether the system is 32-bit or 64-bit, the operating system version and the full browser version string.

Languages

The browser exposes through `navigator.languages` the user's language preferences as a BCP 47 tag list ordered by priority (e.g., `"en-GB"`). Web applications use this to serve content in the appropriate language and format dates, numbers and currencies according to local conventions. The combination of language and region subtags reveals the user's probable nationality and location. For example, a primary language of `ro-RO` almost certainly indicates a Romanian speaker.

Hardware

Through `navigator.hardwareConcurrency`, the browser provides the number of logical CPU cores. This information helps applications start a suitable number of Web Workers for parallel computation. The `deviceMemory` property gives an estimate of the device's RAM in gigabytes. This feature supports better content delivery. Although each value alone is not unique, combining them with other factors significantly reduces the population.

Screen

The **Screen** object exposes the monitor resolution (`screen.width`, `screen.height`), the usable area after operating system chrome like taskbars (`screen.availWidth`, `screen.availHeight`) and the colour depth (`screen.colorDepth`). `window.devicePixelRatio` reports the ratio between physical and CSS pixels:

- 1 = standard displays
- 2 = HiDPI/Retina screens

- non-integer (e.g. 1.25, 1.5) = scaled displays

These properties exist because responsive web design, CSS media queries and canvas rendering all depend on knowing the display characteristics.

Timezone

The browser exposes the user's local timezone through two channels. The first (`Date.prototype.getTimezoneOffset()`) returns the numeric difference in minutes between the user's local time and Coordinated Universal Time (UTC). The second (`Intl.DateTimeFormat().resolvedOptions().timeZone`) returns a named identifier that specifies not just the offset but the exact geographic region, including its daylight saving time rules.

Beyond these explicit properties, every method that returns a local time implicitly uses the timezone. Websites rely on these APIs to show correct local times in calendars, booking systems and timestamps. The IANA identifier is one of the strongest geolocation signals available through JavaScript, as it identifies not just a UTC offset but a specific geographic region with its own daylight saving time rules.

Canvas

The HTML5 Canvas API provides a 2D drawing surface that scripts can use to render text, shapes, and images programmatically. A canvas fingerprint is generated by drawing a specific scene, typically text in a particular font alongside geometric shapes, and reading back the pixel data using `toDataURL()` or `getImageData()`. The same drawing commands produce subtly different pixel output on different systems due to differences in font rasterisation (FreeType, DirectWrite, Core Text), GPU rendering pipelines (Direct3D, Metal, OpenGL) and colour management. The canvas API is essential for data visualisation, browser games, image editing, PDF rendering, etc., making it impossible to remove without breaking a large portion of the web.

WebGL

WebGL is the browser API for hardware-accelerated graphics rendering. It includes a debugging extension that exposes the exact name of the GPU vendor and the full model identifier of the graphics card. These strings are detailed enough to identify not just the manufacturer but the specific GPU model, generation and, on Windows, the rendering pipeline and shader version in use. This extension was intended for debugging driver issues, but because users rarely change their graphics card, it provides a persistent, high-entropy identifier.

Audio

The Web Audio API processes audio through a graph of `AudioNode` objects. Fingerprinting scripts use it by constructing a small audio processing graph — a signal generator feeding into a dynamics processor — and reading the raw numerical output without ever playing any sound. The Web Audio API handles audio using a graph of `AudioNode` objects. Fingerprinting scripts create a simple audio processing graph — a signal source connected to a dynamics processor — and obtain raw numerical data without producing any audible sound. The results rely on the exact floating-point behaviour of the processor

and the browser's audio library. These variations, which differ slightly across hardware and software setups, are too small to hear but are consistent and measurable.

WebRTC

WebRTC is the browser technology that enables direct peer-to-peer communication between users, used for video calls, voice chat and file sharing. To establish a connection, the browser must discover all possible network paths between the two peers, which requires collecting the device's network addresses, including both the local address assigned by the home router and the public address visible on the internet.

The local IP address is particularly dangerous because it persists across sessions, survives cookie clearing, and remains visible even when the user is connected through a VPN. The VPN changes the public IP, but the local network address assigned by the router is still included in host candidates.

Speech Synthesis

The `speechSynthesis.getVoices()` method returns an array of available text-to-speech voices, each with a name, language tag and a flag indicating whether it runs locally or using a cloud service. The set of voices varies significantly between systems depending on the operating system, its version, installed language packs, and third-party TTS (Text-to-Speech) software.

4.2. Two-Layer Architecture

Fingerprinting data is collected during two distinct phases of a page load. If the defense addresses only one of them, the other remains exposed.

4.2.1. Layer 1: HTTP Headers (Network)

Every HTTP request that a browser sends carries metadata in its headers. Some of the metadata included is the **User-Agent string**, the **Accepted-Language** preference list, and the **Sec-CH-UA Client Hints** fields. When the first request is made, a default set of **Sec-CH-UA*** headers is sent, including **Sec-CH-UA-Platform** and **Sec-CH-UA-Mobile**. The other headers are available to the server, and it can request them, but to keep the header light and not overly loaded, a shorter version is used.

A tracking server can read and store these header values before any JavaScript runs on the page. This means a defense that only works at the JavaScript level has no effect on this type of collection.

4.2.2. Layer 2: JavaScript APIs

Once the page loads, fingerprinting scripts access various browser APIs to collect device-specific data. This includes simple details such as **screen resolution** and **time-zone**, as well as more advanced information like **canvas** renderings, **GPU** details, **audio** processing results, and local **IP addresses** revealed through WebRTC. Modifying HTTP headers alone cannot change any of these values.

4.2.3. Synchronization between Layers

An effective defense must, therefore, operate concurrently at both layers. However, merely modifying each layer independently is insufficient, the values reported by both layers must be in agreement.

This means the defense model will require three components:

- A **network layer component** that modifies the HTTP headers of responses prior to reaching the server, ensuring that the server used for fingerprinting sees the substituted identity.
- A **JavaScript-layer component** that intercepts browser API calls and returns substitute values, so fingerprinting scripts on the page also see the fake identity instead of the real one.
- A **synchronization mechanism** between the two layers, ensuring that the profile chosen at random is applied consistently across both HTTP headers and JavaScript APIs.

Figure 4.2 illustrates the described model. When the extension first loads, the JavaScript-layer component selects a profile and communicates it to the network-layer component through the synchronization channel.

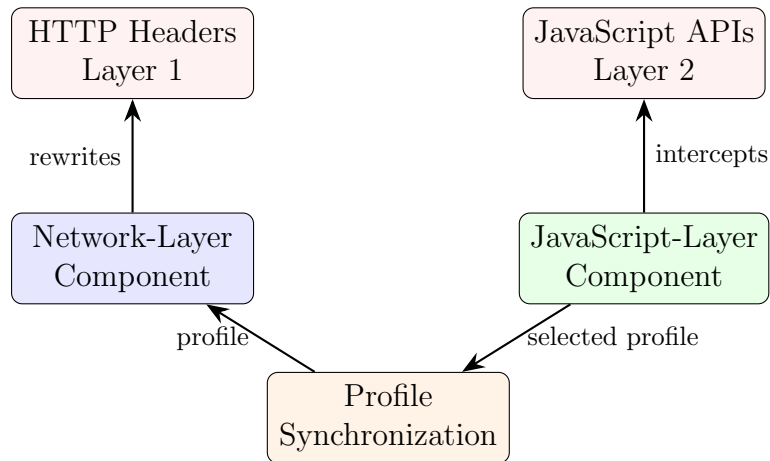


Figure 4.2: Abstract two-layer defense model

4.3. Values Substitution Mechanism

4.3.1. Consistency vs. Diversity

The defense must satisfy two properties that may seem contradictory at first:

1. **Consistency Within a Session:** Every API query during a browsing session should consistently reflect a coherent profile. If the user-agent string indicates Chrome 142, then the Client Hints array, HTTP User-Agent header, and navigator.userAgentData must all confirm version 142. Similarly, the reported screen resolution, language preferences, timezone, and GPU model should form a plausible combination for a real device. Any internal inconsistency, such as a Linux platform string paired with

a Windows user-agent, may suggest the values are not real.

2. **Diversity Across Sessions:** Across different browsing sessions, unique profile must be generated. If every extension user always received the same substitute identity, the defense would effectively create a static fingerprint shared across all users. Fingerprinting services could quickly identify this profile and treat it as one entity. Ideally, each session should present distinct profile.

4.3.2. Pseudo-Random Number Generator

The previously mentioned properties can be achieved with a simple principle: using a pseudo-random number generator (PRNG). It takes a seed string as input and produces a deterministic sequence of floating-point numbers in the range $[0, 1)$. All profile selections are derived from this sequence, ensuring that the same seed always produces the same fingerprint. Since the algorithm is deterministic, the same seed always produces the same set of values, which satisfies consistency. Because the seed is randomly generated, different sessions receive different seeds and therefore different profiles, which satisfies diversity.

4.3.3. Session Seed

The session seed is generated by concatenating two calls to `Math.random()`, each converted to a base-36 string with the leading "0." prefix removed:

$$\text{seed} = r_1.\text{toString}(36).\text{slice}(2) \parallel r_2.\text{toString}(36).\text{slice}(2) \quad (4.1)$$

Since each call to `Math.random()` generates a 52-bit mantissa and base-36 encoding provides roughly 5.17 bits per character, each call results in about 11 alphanumeric characters. Combining two such calls yields a string of around 22 characters (114 bits of entropy), making seed collisions between separate sessions extremely unlikely in practice. The seed is stored in `sessionStorage` on the first access and retrieved during later page navigations within the same tab, maintaining a consistent fingerprint profile throughout the session. If `sessionStorage` cannot be accessed, a fallback seed is created for the current page load only.

4.3.4. Mulberry32 Hash-Based Generator

The PRNG is based on the **Mulberry32 algorithm**, a fast 32-bit hash-based generator. The implementation derives two independent 32-bit state variables from the seed string using polynomial rolling hashes with different multipliers (31 and 37):

$$a_0 = \sum_{i=0}^{n-1} 31^{n-1-i} \cdot c_i \mod 2^{32}, \quad b_0 = \sum_{i=0}^{n-1} 37^{n-1-i} \cdot c_i \mod 2^{32} \quad (4.2)$$

- where c_i is the Unicode code point of the i -th character of the seed string
- all arithmetic is performed modulo 2^{32} (using JavaScript's bitwise OR with zero to truncate to 32 bits)

The two state variables provide 64 bits of internal state, reducing the probability of collisions when the PRNG is used for fine-grained noise generation (canvas pixels, audio samples).

Each invocation advances the state by adding Weyl sequence constants and applies two rounds of xorshift-multiply to produce a uniform output:

$$a_{k+1} = a_k + 0x6D2B79F5 \mod 2^{32} \quad (4.3)$$

$$b_{k+1} = b_k + 0x9E3779B9 \mod 2^{32} \quad (4.4)$$

$$t = (a_{k+1} \oplus (a_{k+1} \gg 15)) \times (1 \mid b_{k+1}) \quad (4.5)$$

$$t = (t + (t \oplus (t \gg 7)) \times (61 \mid t)) \oplus t \quad (4.6)$$

$$\text{output} = \frac{(t \oplus (t \gg 14)) \mod 2^{32}}{2^{32}} \quad (4.7)$$

The constant `0x9E3779B9` is the integer part of $2^{32}/\varphi$ (where φ is the golden ratio), a well-known constant in hash function design that provides good distribution of the Weyl sequence.

4.3.5. Separate PRNG Instances

The extension creates three independent PRNG instances from the same base seed, each with a different suffix:

- **Main PRNG** (seed): used for all profile selections (UA, screen, GPU, language, timezone, memory, CPU cores).
- **Canvas PRNG** (seed + “_cv”): used for canvas colour perturbation and font substitution.
- **Audio PRNG** (seed + “_aud”): used for audio buffer noise injection.

The suffix ensures that the three generators produce independent sequences, so that the canvas perturbation does not consume values from the main profile selection stream, and vice versa. This separation is important because the canvas and audio PRNGs are called thousands of times per page (once per pixel or audio sample), while the main PRNG is called fewer than 20 times per session.

4.4. Value Pool Strategy

The **value pools** consist of predefined arrays containing realistic, real-world values for each fingerprinting vector such as UA strings, GPU profiles, screen resolutions, languages, timezones, memory sizes, and CPU cores. Instead of generating random or obviously fake data, each pool is built using data from actual usage statistics (StatCounter [8], Steam Hardware Survey [9]). This approach ensures that spoofed values appear authentic and do not raise concerns about their validity. Each property has about 5-6 pool size options.

4.4.1. Pool Construction

- **Browser versions** are drawn from the StatCounter statistics, covering the 5 most used Chrome versions. Only recent versions are included, because older versions

are statistically rare and would draw attention.

- **GPU identifiers** are taken from the Steam Hardware Survey, which publishes monthly statistics on the graphics cards used by its large user base. The identifiers are formatted in the ANGLE notation that Chrome uses internally (e.g., ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)).
- **Memory and CPU** values represent typical desktop configurations. Rather than listing every potential value, the pool includes a small selection of common options. (8, 16, 32 GB for memory; 4, 6, 8, 10, 16 for CPU cores).
- **Languages** are European language tags (e.g. en-GB) representing a realistic browsing population.
- **Timezones** are Western and Central European Interned Assigned Numbers Authority (IANA) zone names with their corresponding UTC offsets.
- **Screen resolutions** are the four most common desktop monitor sizes.
- **Fonts** are system fonts typically pre-installed on Windows, used for canvas text substitution.

4.4.2. Weighting Through Repetition

Some values are far more common in real world than others. If every entry in pool had equal probability of selection, rare configurations would be over-represented among extension users, creating a statistical anomaly.

The pools employ a straightforward weighting method: common values are duplicated multiple times within the array. The distribution of values in the pools aims to mirror the real-world distribution of the user's information, striving to create a user profile that closely resembles reality.

Table 4.2: Value pools and their weights

Pool	Size	Data Source
UA_PROFILES	6	Recent Chrome releases (versions 137–142) on Windows 10 x64
WEBGL_PROFILES	7	GPU identifiers from Steam Hardware Survey market share data
MEMORY_POOL	7	Weighted: 16 GB ($\times 3$, 43%), 32 GB ($\times 3$, 43%), 8 GB ($\times 1$, 14%)
CPU_POOL	9	Weighted: 6 cores ($\times 3$, 33%), 8 cores ($\times 3$, 33%), 4/10/16 ($\times 1$ each, 11%)

Table 4.2 *Value pools and their weights*

Pool	Size	Data Source
LANGUAGE_POOL	7	European browser language preferences (en-GB, en-US, es, de, fr, it)
TIMEZONE_POOL	5	Western/Central European IANA zones (Berlin, Paris, London, Warsaw, Rome)
SCREEN_PROFILES	4	Common desktop resolutions (1920×1080, 1536×864, 1366×768, 2560×1440)
CANVAS_FONT_POOL	12	Fonts typically available on Windows installations

4.4.3. Session Consistent Identity

All pools are selected exactly once per page load, right after reading the seed from session storage and initializing the pseudo-random number generator. The generator picks one entry from each pool, forming the page’s substitute profile. All later API queries during that page’s lifetime return values from this profile.

Because the generator is deterministic and the pools are fixed, a given seed will always produce the same selection. Since the seed remains stored in session storage for the duration of the tab, the same profile is reconstructed on each navigation within the session, ensuring consistency.

4.5. Interception Strategies

After selecting a profile, it is crucial to ensure that the browser’s built-in APIs provide substitute values rather than the actual ones. This step is particularly challenging because the replacement must be invisible: the APIs need to function exactly as they do in a normal, unmodified browser, except for returning different data.

There are three primary methods employed in the implementation: to replace the user’s real profile with the session profile and to hide any trace that the returned values have been altered.

4.5.1. Prototype Property Replacement

JavaScript employs a prototype-based inheritance model instead of the conventional class-based approach. Each function has a **prototype** property and every object maintains an internal reference to a prototype object (**Constructor.prototype**). Rather than copying properties from a class blueprint, objects delegate property lookups to this shared prototype. When a property is accessed, the engine first inspects the object itself. If the property is not present, it follows the internal prototype link and searches on the prototype object (**Class.prototype**). This process repeats along the prototype chain until the property is found or the chain ends at **null**.

This is the core of the first strategy: replace the retrieval function in the shared template with one that provides a selected value rather than the actual one. Any script

accessing that property will receive the substitute, with no visible difference in how the access behaves.

The strategy is effective as long as scripts only read the value. Problems occur when a script does not trust the value and inspects the template directly to verify that the retrieval function is the original one installed by the browser. Browser functions have a mark identifying them as built-in, which scripts can check. If a replacement is detected instead, the script can conclude that interception has occurred.

4.5.2. Proxy-Based Interception

Some of the properties/vectors cannot be simply replaced because they are easy to detect.

Among browser APIs, `Navigator` is the most complex and targeted by fingerprinting tools because its prototype contains many properties related to the user's environment, such as user agent, platform, language, plugin list, hardware info and device memory. The data from these fields can be easily combined to create a unique identifier.

Furthermore, fingerprinting scripts do not just read the `Navigator` properties, they also examine the prototype's structure itself. Calling `getOwnPropertyNames(Navigator.prototype)` returns an ordered array of all properties defined on the prototype. Also, a full descriptor of each entry, including whether the getter is native or a JavaScript replacement, can be retrieved by calling `getOwnPropertyDescriptor(...)`. Because fingerprinting scripts inspect the prototype this deeply, simply replacing a property on the prototype is not a viable defence, especially due to a behaviour specific to Chrome's V8 engine.

V8 does not immediately attach all getters to `Navigator.prototype` upon the initial page load. Instead, it uses **lazy installation**: the native getter for a property is only added to the prototype the first time a script accesses that property. This is an internal optimization aimed at avoiding the overhead of setting up getters for properties that may never be used during the page's lifetime. Consequently, this introduces a timing issue. If the defence defines a replacement property before any code has triggered the lazy installation of the native getter, the interpreter does not recognise it as a replacement, it sees it as a new property. According to the **ECMAScript** specification, newly created properties are positioned at the end of the object's internal property order.

The solution is to use an ES6 Proxy object. Instead of modifying `Navigator.prototype`, the `navigator` instance is wrapped in a Proxy. The Proxy's `get` trap intercepts property reads and returns substitute values for known properties, while delegating everything else to the real navigator object. Crucially, the Proxy's structural traps (`ownKeys`, `getOwnPropertyDescriptor`, `has`) forward directly to the real object, so any reflective inspection sees the native prototype with its original property order, native descriptors, and native getter identities.

4.5.3. `toString()` Override

When a JavaScript function is replaced, its source code changes. Calling `func.toString()` on a native browser function returns the string `function name() { [native code] }`, but calling it on a replacement function reveals the actual JavaScript source. Fingerprinting detection tools exploit this difference to identify modified APIs.

This is addressed by intercepting `Function.prototype.toString` method, which all functions use to produce their string representation. Each replacement function is

stored in a `Map` alongside the name of the native function it replaces. When `toString()` is called on any of these functions, the intercepted method looks up the original name and returns a string matching the expected native code format.

Chapter 5. Detailed Design and Implementation

This chapter documents the concrete implementation of the strategies described in Chapter 4. It covers the extension structure, the Manifest V3 configuration, the source code of key functions and implementation details, and the inter-component communication mechanism.

5.1. Extension Architecture

The extension is four source files running across three execution contexts. Figure 5.1 shows them and how they communicate.

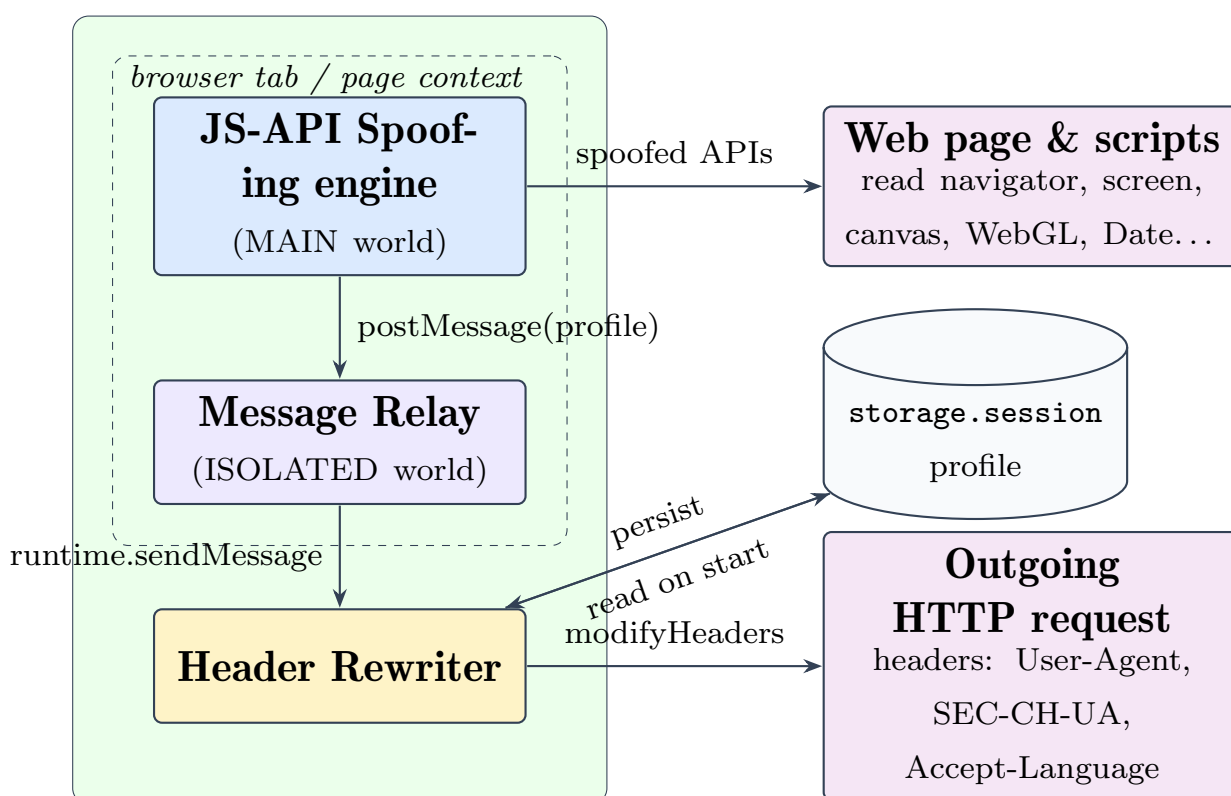


Figure 5.1: Extension architecture

Chrome’s content script system defines two execution *worlds* that determine the scope and visibility of a script’s execution.

The **MAIN world** is the primary JavaScript execution environment of a webpage, where all page scripts run and share a single global scope (window). Content scripts can be explicitly configured to run in this environment.

The **ISOLATED world** is a separate, sandboxed execution environment created by the browser for extension content scripts, which shares access to the same Document Object Model (DOM) but maintains an entirely independent JavaScript scope.

Manifest File (`manifest.json`)

The manifest file is the extension's configuration file that tells Chrome what permissions it needs, which scripts to load, and where and when to run them.

It declares the three permissions the extension requires (`declarativeNetRequest`, `declarativeNetRequestFeedback` and `storage`), the background service worker and the two content scripts together with the execution *world* each runs in. Both content scripts are registered with `run_at: document_start`, `all_frames: true`, and `match_about_blank: true`, so they are injected before any page JavaScript executes and cover sub-frames as well as the top-level document.

Table 5.1: Key `manifest.json` settings

Setting	Purpose
<code>"world": "MAIN"</code>	Injects content script into the page's own JavaScript environment so API replacements affect page scripts
<code>"run_at": "document_start"</code>	Ensures replacements are installed before any page script runs
<code>"all_frames": true</code>	Injects into every iframe, not just the top-level frame
<code>"match_about_blank": true</code>	Covers dynamically created <code>about:blank</code> frames
<code>declarativeNetRequest</code>	Permission to alter HTTP headers using rules
<code>storage</code>	Permission for <code>chrome.storage.session</code> (profile synchronization)

Spoofing Engine (`content.js`)

It is the core JavaScript-API spoofing engine. It runs in the page's **MAIN** world (the standard JavaScript environment provided by the web browser for each webpage) at `document_start`, giving it direct access to the page's global objects before any site script can read or cache them. On load, it generates a stable session seed from `sessionStorage` and uses it to initialize a Mulberry32 seeded PRNG. The implementation of the PRNG algorithm is presented below:

Listing 5.1: Mulberry32 PRNG implementation (`content.js`)

```

1 function makePRNG(seedStr) {
2   let a = 0, b = 0;
3   for (let i = 0; i < seedStr.length; i++) {
4     const c = seedStr.charCodeAt(i);
5     a = Math.imul(31, a) + c | 0;
6     b = Math.imul(37, b) + c | 0;
7   }

```

```

8   return function () {
9       a |= 0; a = a + 0x6D2B79F5 | 0;
10      b |= 0; b = b + 0x9E3779B9 | 0;
11      let t = Math.imul(a ^ a >>> 15, 1 | b);
12      t = t + Math.imul(t ^ t >>> 7, 61 | t) ^ t;
13      return ((t ^ t >>> 14) >>> 0) / 4294967296;
14  };
15  }

```

Three independent instances are generated from the seed - `rand`, `_cvRand`, and `_audRand` - and one entry is chosen from each value collection to create the session profile. Table 5.2 shows how each pool corresponds to the API surface it provides.

Table 5.2: Value pools, the API surfaces and an example entry

Pool	Drives	Example value
UA_PROFILES	<code>userAgent</code> , Client Hints brands, platform version	Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/142.0.0.0
WEBGL_PROFILES	WebGL unmasked vendor and renderer strings	ANGLE (NVIDIA, GeForce RTX 3060 Direct3D11, D3D11)
MEMORY_POOL	<code>navigator.deviceMemory</code>	16 (GB)
CPU_POOL	<code>navigator.hardwareConcurrency</code>	8 (cores)
LANGUAGE_POOL	<code>navigator.language(s)</code> , <code>Accept-Language</code>	["de-DE", "de", "en-GB", "en"]
TIMEZONE_POOL	<code>Date</code> / Intl timezone and offset	zone: Europe/Berlin, offset: -120
SCREEN_PROFILES	<code>screen.*</code> , <code>outerWidth/Height</code> , <code>availHeight</code>	width: 1920, availWidth: 1920
CANVAS_FONT_POOL	substitute font family in the canvas hook	Times New Roman

Next, it replaces all APIs in the page's JavaScript environment that are targeted by fingerprinting, forwarding the selected profile to `background.js` through `bridge.js` for synchronisation of HTTP headers.

Message Relay (`bridge.js`)

This minimal content script runs within the `ISOLATED` world. Since content scripts in the `MAIN` world cannot access the `chrome.*` extension APIs, `content.js` must communicate with the service worker through an intermediary. `bridge.js` serves this purpose: it listens for `message` events from the same window (preventing messages from embedded frames or remote origins). When it detects an `__AFP_UPDATE_PROFILE__` event, it forwards the payload to the service worker using `chrome.runtime.sendMessage`.

Header Rewriter (`background.js`)

It is the service worker responsible for HTTP-layer header rewriting. On startup, it calls `createRules()`, which reads the active profile from `chrome.storage.session`; if none has been stored yet it falls back to a hardcoded Chrome 142 default. From the

profile it builds a `Sec-CH-UA` header by formatting the brands array into the structured-field syntax required by the Client Hints specification, and an `Accept-Language` header with descending *q*-values (`en-GB,en;q=0.9`, etc.). It then calls `declarativeNetRequest.updateDynamicRules` to install a single rule that sets `User-Agent`, `Sec-CH-UA`, `Sec-CH-UA-Mobile`, `Sec-CH-UA-Platform`, and `Accept-Language` on every outgoing request, covering all resource types. When a profile-update message arrives from `bridge.js`, the service worker stores the new profile in `chrome.storage.session` and calls `createRules()` again so the HTTP rules stay in sync with the JavaScript-layer identity chosen by `content.js`.

5.2. Profile Initialization and Synchronization Sequence

Figure 5.2 illustrates the internal initialization sequence that occurs on each page load. Chrome injects `content.js` at `document_start` before any site script runs. This script retrieves or creates a session seed from `sessionStorage`, feeds it into a Mulberry32 PRNG, and selects a stable value from each pool — user-agent, GPU, memory, CPU, language, timezone, and screen — to form a session profile. It then installs hooked versions of the addressed APIs in the page’s JavaScript environment. The profile is sent through `bridge.js` to `background.js`, which stores it and updates the `declarativeNetRequest` rules so that outgoing HTTP headers match the JavaScript-layer identity.

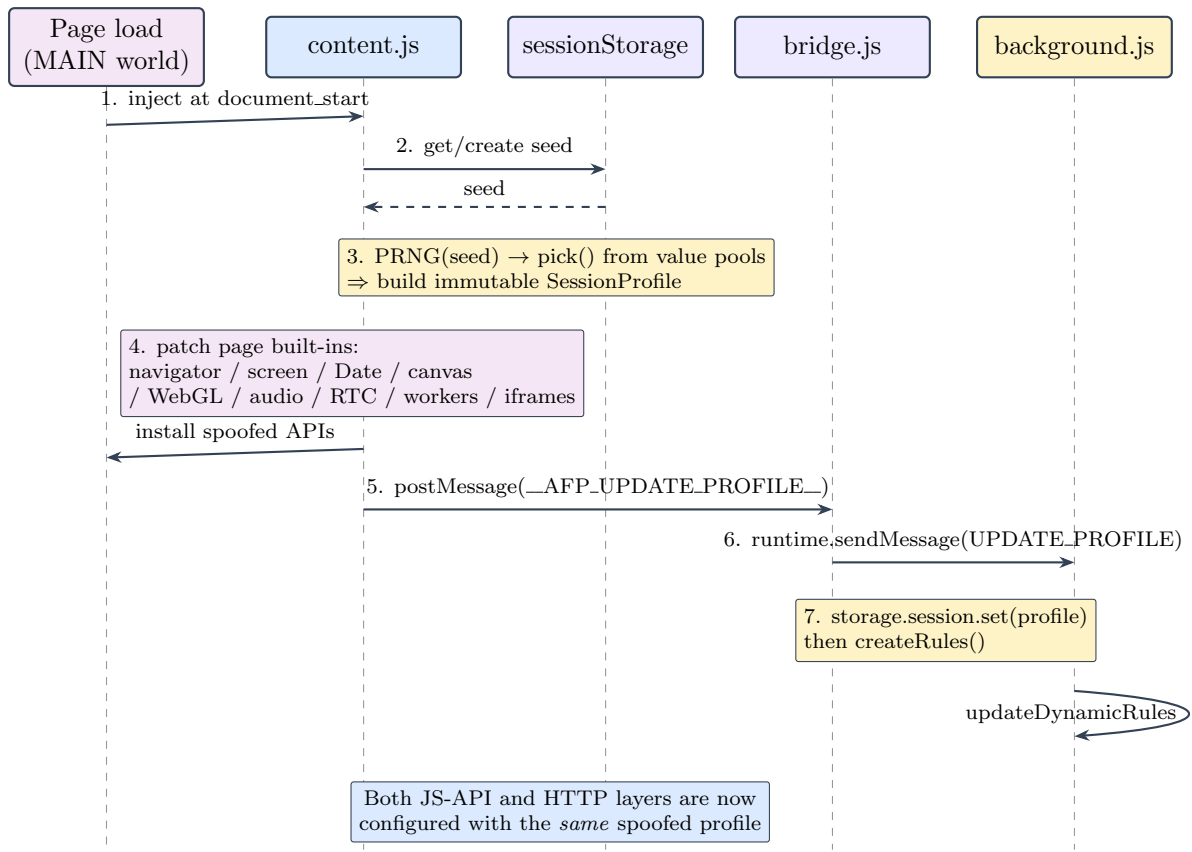


Figure 5.2: Profile initialization and synchronization sequence

Figure 5.3 shows what happens at runtime when a user visits a website whose server deploys fingerprinting. The user’s browser sends a page request whose HTTP headers are

rewritten by the header rewriter before leaving the browser, so the server already receives a spoofed identity in the initial request. The server responds with a page that includes fingerprinting scripts. When these scripts execute, any JavaScript API read — such as `navigator.userAgent` or `screen.width` — is intercepted by the spoofing engine and returns the fake value. The scripts then send the collected fingerprint back to the server, which receives only spoofed data. As a result, the server has no means of observing the user’s real browser profile through either channel.

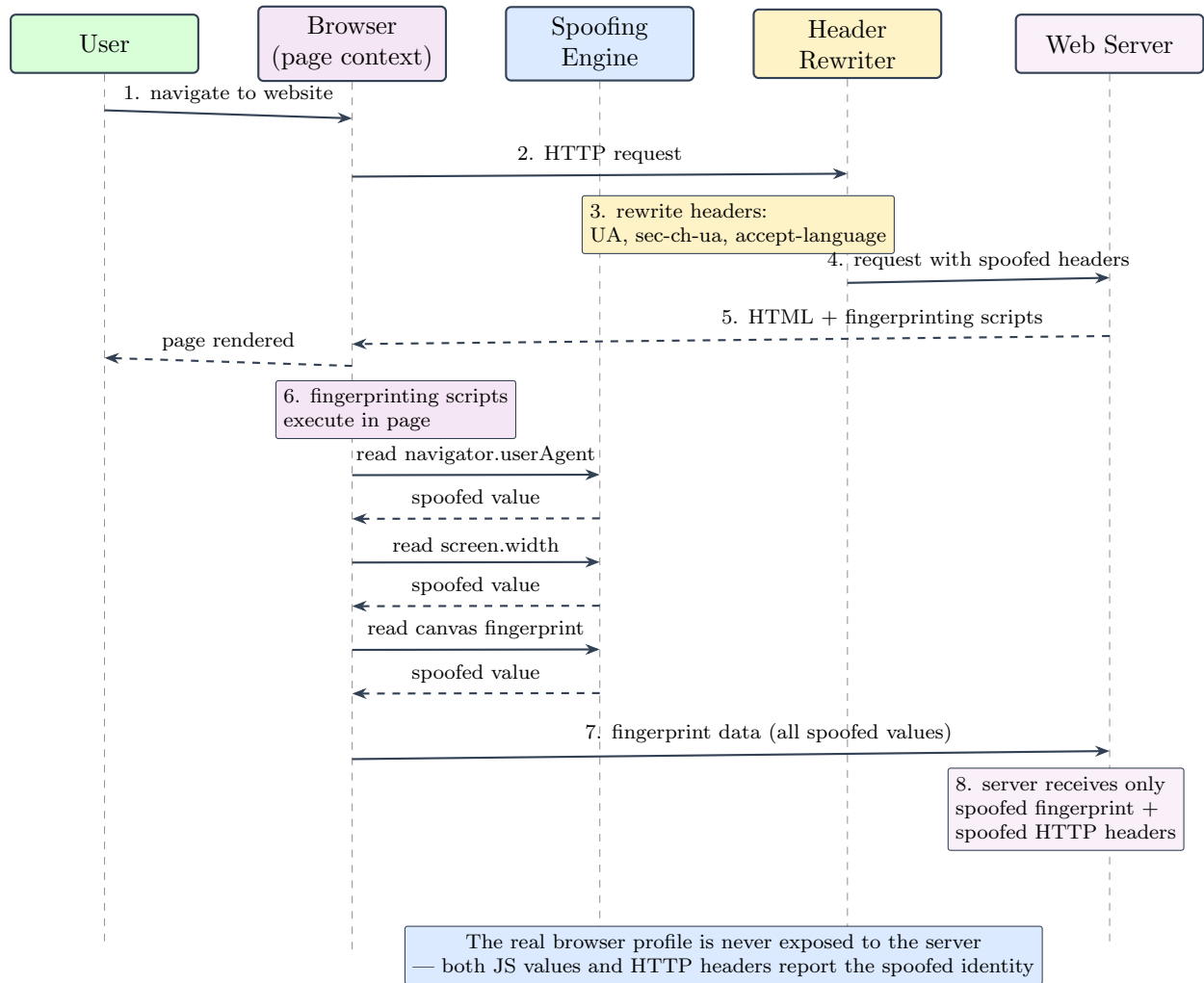


Figure 5.3: Runtime interaction: fingerprinting attempts receive only spoofed data

5.3. Function Masking

Each interception method in this chapter swaps out a native browser function with a JavaScript version. If a fingerprinting script uses `toString()` on these substitutes, it gets the real JavaScript source code of the function rather than the typical `function name() { [native code] }` string that genuine browser functions return. Detection tools take advantage of this distinction to spot altered environments before inspecting any property values.

To prevent this, the extension saves the real `Function.prototype.toString` and replaces it with a wrapper that checks three cases in order:

- if `toString` is called on itself, it returns its own native-looking string to avoid infinite recursion
- if the target function is registered in a `hookedFunctions` Map, it returns a native-looking string using the original function's name
- otherwise it delegates to the saved real implementation, leaving all non-hooked functions completely unaffected

Each replacement function is registered in the Map immediately after its definition using a `hideFn` helper. This helper also redefines the function's `name` property using `Object.defineProperty` method. Normalizing `name` alongside `toString` is a cost-effective hardening measure against any check that also inspects it, though the `toString` comparison is the primary signal.

Listing 5.2: Function masking implementation

```

1  const hookedFunctions = new Map();
2  const originalToString = Function.prototype.toString;
3
4  Object.defineProperty(Function.prototype, 'toString', {
5    value: function toString() {
6      if (this === Function.prototype.toString)
7        return "function_␣toString()␣{␣[native_␣code]␣}";
8      if (hookedFunctions.has(this))
9        return `function ${hookedFunctions.get(this)}() { [native code]
10         }`;
11      return originalToString.call(this);
12    },
13    configurable: true, writable: true,
14  });
15
16  const hideFn = (fn, name) => {
17    hookedFunctions.set(fn, name);
18    Object.defineProperty(fn, 'name', { value: name, configurable: true });
19  };

```

5.4. JavaScript API Hooking Implementation

This section documents how each fingerprinting vector is intercepted in `content.js`, following the interception strategies described in Chapter 4.

5.4.1. Navigator

This JavaScript object is built into all browsers, providing details about the browser and device. Accessible via `extttwindow.navigator` to scripts on any webpage, it was initially created to help websites tailor content and behavior to different browsers and devices. However, since it discloses a wide range of detailed device information, it has become a primary source of fingerprinting data used by tracking scripts.

The `navigator` object is wrapped in an ES6 Proxy rather than patched on its prototype, for the reasons discussed in Chapter 4. A lookup table (`_navSpoofed`) holds

the substitute values for the properties that are overridden and the Proxy's `get` trap intercepts reads of those properties while delegating everything else to the real `navigator`:

Listing 5.3: Navigator Proxy setup

```

1  const _navSpoofed = {
2    userAgent: uaProfile.ua,
3    appVersion: uaProfile.ua.replace("Mozilla/", ""),
4    platform: uaProfile.platform,
5    deviceMemory: memory,
6    hardwareConcurrency: cpuCores,
7    language: languages[0],
8    vendor: "Google Inc.",
9    webdriver: false,
10   userAgentData: _uadObject,
11 };
12
13 const _navigatorProxy = new Proxy(navigator, {
14   get(target, prop) {
15     if (prop === 'languages') return Object.freeze([...languages]);
16     if (Object.prototype.hasOwnProperty.call(_navSpoofed, prop))
17       return _navSpoofed[prop];
18     const val = Reflect.get(target, prop, target);
19     return typeof val === 'function' ? val.bind(target) : val;
20   },
21   ownKeys: (t) => Reflect.ownKeys(t),
22   getOwnPropertyDescriptor: (t, p) =>
23     Reflect.getOwnPropertyDescriptor(t, p),
24   has: (t, p) => Reflect.has(t, p),
25 });
26
27 Object.defineProperty(window, 'navigator', {
28   get: () => _navigatorProxy,
29   enumerable: true, configurable: true,
30 });

```

When a new Proxy is created, it has two parameters: `target`, the object of focus, and `handler`, which specifies how the Proxy behaves and which operations it intercepts. Each interceptable operation corresponds to a *trap*, a function defined on the handler that is called instead of performing the default operation. For instance, the `get` trap is a function that captures every property read on the wrapped object. In the extension, the `get` trap intercepts every property read on `navigator` and returns the corresponding spoofed value when the requested property is one of the overridden ones. The other structural traps (`ownKeys`, `getOwnPropertyDescriptor` and `has`) are forwarded directly to the real object, so reflective inspection of the prototype sees native descriptors and the original property order.

The `userAgentData` object (`_uadObject`) is constructed to mirror the shape of Chrome's `NavigatorUAData` interface, including a `getHighEntropyValues()` method that returns a `Promise`¹ resolving to the full set of Client Hints fields: *brands*, *mobile*,

¹A `Promise` is a JavaScript object representing the result of an asynchronous operation — a placeholder for a value that will be available in the future. `Promise.resolve(value)` creates one that is

platform/platformVersion, *bitness*, *architecture*, *model*. Functions inside the object are registered with `hideFn` so their `toString()` output matches the native format.

5.4.2. Screen and Window

Screen properties are modified by redefining the property descriptors directly on `Screen.prototype` instead of on individual `screen` instances, since all screen objects across different frames share the same prototype. This means a single replacement is visible everywhere without needing to change each instance on every frame. The properties `width`, `height`, `availWidth`, `availHeight`, `colorDepth`, and `pixelDepth` are each given a getter returning the corresponding field from the chosen `SCREEN_PROFILE`.

`colorDepth` and `pixelDepth` are fixed to 24 because:

- 24-bit colour involves 8 bits for each channel (red, green, blue), which has been the standard format and is used by most monitors today
- browsers report 24 even on displays that are technically capable of higher colour depths (such as 30-bit HDR monitors), because the browser colour pipeline operates at 24-bit by default.

Listing 5.4: Screen prototype patching

```

1 const screenDescriptors = {
2   width: { get: () => screenProfile.width },
3   height: { get: () => screenProfile.height },
4   availWidth: { get: () => screenProfile.availWidth },
5   availHeight: { get: () => screenProfile.availHeight },
6   colorDepth: { get: () => 24 },
7   pixelDepth: { get: () => 24 },
8 };
9 Object.entries(screenDescriptors).forEach(([prop, descriptor]) => {
10   Object.defineProperty(Screen.prototype, prop,
11     { ...descriptor, configurable: true });
12 });

```

Three window-level properties — `outerWidth`, `outerHeight` and `devicePixelRatio` — cannot be patched through a shared prototype and are therefore redefined directly on `window` using `Object.defineProperty`.

- `outerWidth` and `outerHeight` represent the total size of the browser window, including the chrome elements like the toolbar, address bar, and borders. If these reflect their actual measurements, a script could compare them with the spoofed `screen.width` and `screen.height` values to detect discrepancies. For instance, an `outerWidth` larger than the reported screen width would be suspicious. To avoid this, they are set to match the selected screen resolution.
- `devicePixelRatio` indicates the ratio between physical and CSS pixels. If it had a **non-default** value, it would show the actual display type regardless of any resolution spoofing. Therefore, it is set to 1, matching the most common display setup and aligning with the selected screen profiles, which all represent standard desktop monitors.

already fulfilled.

5.4.3. Timezone

The timezone section is the most advanced because it involves spoofing multiple interconnected API surfaces, each requiring specific adjustment logic. If only one property had been changed, a fingerprinting script could simply query any one of them to determine the user’s actual geographic region. The main challenge lies in JavaScript’s `Date` object, which stores time internally in UTC but converts it to local time whenever the user accesses or modifies hours, minutes, or formatted date strings.

The implementation replaces six groups of functions, each described in the following subsections.

Intl.DateTimeFormat

The `Intl.DateTimeFormat` API provides a modern method to format dates and times based on the locale. If a script creates a new formatter without specifying a timezone, the browser defaults to the user’s actual timezone. The modification involves replacing the constructor with a wrapper that silently inserts a fake timezone whenever the caller does not specify one.

Listing 5.5: `Intl.DateTimeFormat` wrapper

```

1 const _FakeIntlDTF = function DateTimeFormat(locales, options) {
2   const opts = options ? { ...options } : {};
3   if (!opts.timeZone) opts.timeZone = tzProfile.zone;
4   return new.target
5     ? Reflect.construct(_OrigIntlDTF, [locales, opts], new.target)
6     : _OrigIntlDTF.call(this, locales, opts);
7 };

```

The modified constructor keeps the original prototype and static methods, so that existing code using `instanceof` or `supportedLocalesOf` works the same as before.

The `resolvedOptions()` method has been separately replaced to consistently report the spoofed timezone and locale, even for formatters created prior to the patch. This method can be called on a `DateTimeFormat` object after creation to obtain its current settings, such as locale, timezone, calendar, and numbering system.

getTimezoneOffset

The `getTimezoneOffset()` method returns the difference in minutes between UTC and the user’s local time. For a user in Romania (UTC+2/3), it would return $-120/-180$. The patch replaces this method with one that always returns the offset from the chosen timezone profile:

Listing 5.6: `getTimezoneOffset` patch

```

1 const _fakeGetTimezoneOffset = function getTimezoneOffset() {
2   if (Number.isNaN(this.getTime())) return NaN;
3   return tzProfile.offset;
4 };
5 _OrigDate.prototype.getTimezoneOffset = _fakeGetTimezoneOffset;

```

The original `getTimezoneOffset` is saved before being overwritten, because it is needed later in the `Date` constructor patch to compute the adjustment between the real and spoofed timezones.

Local-Time Getters

Methods such as `getHours()`, `getFullYear()`, and `getMonth()` convert the `Date` object's internal UTC timestamp to the user's local time before returning a value. In an unmodified browser, this conversion uses the real timezone, which would leak the user's location.

The patch modifies each getter to manually convert the value using the spoofed offset. Instead of depending on the browser's internal timezone logic, it adjusts the UTC timestamp by the fake offset and then invokes the corresponding UTC getter, which is independent of any timezone.

Listing 5.7: Local-time getter patching

```

1  const _toSL = (ts) => new _OrigDate(ts - tzProfile.offset * 60000);
2
3  const _localGetters = {
4    getFullYear: d => d.getUTCFullYear(),
5    getMonth: d => d.getUTCMonth(),
6    getDate: d => d.getUTCDate(),
7    ...
8  };
9
10 Object.entries(_localGetters).forEach(([name, fn]) => {
11   const fake = function () {
12     const ts = this.getTime();
13     if (Number.isNaN(ts)) return NaN;
14     return fn(_toSL(ts));
15   };
16   _OrigDate.prototype[name] = fake;
17   hideFn(fake, name);
18 });

```

The helper function `_toSL` (“to spoofed local”) creates a new `Date` shifted by the fake offset in milliseconds. All eight getters follow the same pattern, so they are defined in a map and installed in a loop to avoid code repetition.

Local-Time Setters

Setters such as `setHours()` and `setFullYear()` work in the opposite direction from getters: they take a value expressed in local time and convert it to UTC for internal storage. In an unmodified browser, “local time” means the user's real timezone, which would leak their location.

The patch handles this by first shifting the current timestamp into the chosen timezone, then applying the change using the UTC version of the setter, and finally shifting the result back.

Listing 5.8: Local-time setter patching

```

1  const _localSetters = {
2    setHours: 'setUTCHours',
3    ...
4    setDate: 'setUTCDate',

```

```

5     ...
6   };
7
8   Object.entries(_localSetters).forEach(([name, utcName]) => {
9     const orig = _OrigDate.prototype[utcName];
10    const fake = function(...args) {
11      const shifted = new _OrigDate(
12        this.getTime() - tzProfile.offset * 60000);
13      orig.call(shifted, ...args);
14      this.setTime(shifted.getTime() + tzProfile.offset * 60000);
15      return this.getTime();
16    };
17    _OrigDate.prototype[name] = fake;
18    hideFn(fake, name);
19  });

```

String Formatting Methods

When a `Date` object is converted to a string, the output includes the timezone name and offset. For example, `Date.toString()` produces output in the format:

```
Tue Jun 30 2026 22:14:43 GMT+0200 (Central European Summer Time)
```

The patch replaces `toString()` and `toTimeString()` with implementations that build this string manually using the chosen timezone name and offset from the profile, rather than letting the browser insert the real values:

The three locale methods — `toLocaleString()`, `toLocaleDateString()`, and `toLocaleTimeString()` — are handled differently. Each accepts an optional `options` argument that can include a `timeZone` field. The patch wraps each method to inject the spoofed timezone as the default whenever the caller has not specified one, similar to the `Intl.DateTimeFormat` approach.

Date Constructor

The behavior of the `Date` constructor varies based on the type and number of arguments, and the patch addresses three specific cases.

1. **No arguments or a numeric timestamp:** (e.g. `new Date()` or `new Date(1782850760)`) — passed through without modification, since timestamps are timezone-independent.
2. **ISO 8601 strings^[17]** (e.g. `new Date("2026-06-15T14:00:00Z")`) — passed through without modification, since the specification requires ISO strings to be parsed as UTC.
3. **Multi-argument calls and non-ISO strings** (e.g., `new Date(2026, 5, 15)` or `new Date("May 15, 2026")`) — the browser interprets these in the user's real timezone. The patch computes the difference between the real OS offset and the spoofed offset, then shifts the resulting timestamp by that amount.

The adjustment is computed as follows:

Listing 5.9: Date constructor timezone adjustment

```

1 const temp = Reflect.construct(_OrigDate, args, _OrigDate);
2 const realOff = _realGetTimezoneOffset.call(temp);
3 const adj = (tzProfile.offset - realOff) * 60000;
4
5 return Reflect.construct(_OrigDate, [temp.getTime() + adj], new.target);

```

The saved reference to the original `getTimezoneOffset` method is used here to obtain the real OS offset, since the modified version always returns the spoofed value.

5.4.4. Canvas

Canvas fingerprinting is addressed at three key interception points.

Colour perturbation

The extension replaces the `fillStyle` and `strokeStyle` setters on the canvas drawing context. When a script sets a colour for any drawing operation, the extension first converts the colour value into its raw RGB components. This conversion is necessary because colours can be specified in many formats — named colours like “red”, hex codes like “#FF0000” or function notation like “rgb(255,0,0)” — and the extension needs the numeric values to modify them. The conversion is performed by drawing a single pixel on a hidden canvas and reading back the computed colour. Once the RGB values are known, each channel (red, green, blue) is shifted by ± 1 . These shifts are derived from the canvas PRNG instance and remain constant for the entire session. This means that setting the same colour multiple times always produces the same slightly different colour.

Font substitution

The canvas `font` setter is replaced with a version that changes the font family to a different one selected from a pool of 12 common fonts, `CANVAS_FONT_POOL`. The font size and unit specified by the caller are preserved and only the font name is swapped. Since different fonts produce different character shapes and spacing, this changes the rendered text and therefore the resulting canvas hash.

Pixel-level noise in toDataURL

As an additional safeguard, the extension intercepts `HTMLCanvasElement.prototype.toDataURL`, which is used by scripts to export the canvas as an image. When invoked, it makes a temporary copy of the canvas, reads all pixels, applies the same ± 1 RGB shifts used in colour perturbation to each non-transparent pixel, updates the pixels with these modifications, and then exports the altered copy. This ensures protection even when colour and font hooks are insufficient, such as when scripts draw shapes directly with raw pixel data instead of using the `fillStyle` setter.

Listing 5.10: `toDataURL` pixel noise injection

```

1 const _origToDataURL = HTMLCanvasElement.prototype.toDataURL;
2 HTMLCanvasElement.prototype.toDataURL = function(type, quality) {
3     ...
4     const tmp = document.createElement('canvas');
5     tmp.width = w; tmp.height = h;

```

```

6      const tctx = tmp.getContext('2d');
7      tctx.drawImage(this, 0, 0);
8
9      const id = tctx.getImageData(0, 0, w, h);
10     const d = id.data;
11     for (let i = 0; i < d.length; i += 4) {
12         if (d[i + 3] === 0) continue; // skip transparent pixels
13         d[i] = Math.max(0, Math.min(255, d[i] + _colorDeltaR));
14         d[i + 1] = Math.max(0, Math.min(255, d[i + 1] + _colorDeltaG));
15         d[i + 2] = Math.max(0, Math.min(255, d[i + 2] + _colorDeltaB));
16     }
17     tctx.putImageData(id, 0, 0);
18     return _origToDataURL.call(tmp, type, quality);
19 };

```

If any step fails during this process, the extension falls back to calling the original method on the unmodified canvas, preserving normal browser behaviour.

5.4.5. WebGL

`WebGLRenderingContext.prototype.getParameter` and its WebGL 2 counterpart are replaced to intercept two extension-specific parameter codes:

- `UNMASKED_VENDOR_WEBGL` (0x9245)
- `UNMASKED_RENDERER_WEBGL` (0x9246)

Both return the corresponding strings from the chosen `WEBGL_PROFILE`. All other parameter codes are forwarded to the original implementation unchanged.

5.4.6. Audio

Audio fingerprinting is countered by injecting small amounts of noise into the audio buffer before a fingerprinting script can read it. The extension replaces two methods that provide access to audio data: `getChannelData` and `copyFromChannel`. A key challenge is ensuring that reading the same audio channel multiple times always returns the same modified data. If noise were added on every read, the values would change each time. To prevent this, the extension uses a `WeakMap` that tracks which channels have already been modified. Once noise has been applied to a channel, subsequent reads return the same modified data without adding further noise.

The noise is not applied to the entire buffer. Instead, a randomly selected contiguous segment covering 70% to 90% of the buffer is chosen. Near the segment edges, the noise gradually diminishes to prevent sharp transitions that might be noticeable. Inside this segment, individual samples are shifted by about 10^{-5} , a magnitude too small for human hearing, but sufficient to alter the floating-point values used by fingerprinting scripts to generate device-specific hashes.

5.4.7. WebRTC

The extension replaces the `RTCPeerConnection` constructor with a wrapper that forces all connections to use relay-only mode. When a script creates a new peer connection, the wrapper adds `iceTransportPolicy: 'relay'` to the configuration before

passing it to the real constructor. This tells the browser to only use relay servers (TURN servers) for communication and skip the discovery of local and public IP addresses entirely.

Without this restriction, the browser would gather two types of network candidates that reveal identifying information: **host candidates**, which expose the device's local IP address (e.g. 192.168.1.5) and **server-reflexive candidates**, which expose the public IP address visible to the internet. By restricting the transport policy to relay-only, neither type is ever generated.

The real constructor is called internally using `Reflect.construct`, so that the returned object still passes `instanceof RTCPeerConnection` checks and behaves normally in all other respects.

5.4.8. Speech Synthesis

The extension replaces `getVoices` on the `SpeechSynthesis` prototype with a version that filters the browser's native voice list. Only voices whose language tag matches one of the languages in the spoofed profile are returned. For example, if the session profile selects `["de-DE", "de", "en-GB", "en"]`, only German and English voices are included in the result. This prevents a fingerprinting script from enumerating the full set of installed language packs, which would reveal the user's real operating system locale.

5.5. Worker Context Propagation

Web Workers run in their own separate JavaScript environment with their own global object (`self`), completely independent from the main page. This means that all the modifications the extension applies to the main page, such as replacing Navigator properties, patching Date methods, modifying Canvas functions, do not exist inside a Worker. Without additional measures, a fingerprinting script could simply create a Worker, read `navigator.userAgent` or `Date.getTimezoneOffset()` from inside it and get the real unmodified values.

To prevent this, the extension intercepts the `Worker` and `SharedWorker` constructors. Instead of letting the browser load the original Worker script directly, the extension builds a new script that starts with all the spoofed values embedded as constants, followed by the original script loaded through `importScripts()`. This way, by the time the original Worker code runs, the spoofed values are already in place.

Listing 5.11: Worker blob injection

```

1 function makeSpoofedBlobURL(scriptURL) {
2   const absoluteURL = String(new URL(scriptURL, location.href));
3   const blob = new Blob(
4     [WORKER_SPOOF_SCRIPT + '\nimportScripts('
5       + JSON.stringify(absoluteURL) + ');'],
6     { type: 'text/javascript' }
7   );
8   return URL.createObjectURL(blob);
9 }
10
11 const FakeWorker = function Worker(scriptURL, options) {
12   if (options && options.type === 'module')
```

```

13     return new OrigWorker(scriptURL, options);
14     if (!_workerNeedsInjection(scriptURL))
15         return new OrigWorker(scriptURL, options);
16     try {
17         const blobURL = makeSpoofedBlobURL(scriptURL);
18         const worker = new OrigWorker(blobURL, options);
19         URL.revokeObjectURL(blobURL);
20         return worker;
21     } catch (e) { return new OrigWorker(scriptURL, options); }
22 };

```

The spoofed values are embedded directly into the Worker script as JSON constants at the moment the main thread creates it. This makes the Worker entirely self-contained — it does not need to receive values through `postMessage` or any other communication channel. Once the Worker has been created, the temporary blob URL used to load it is revoked immediately to free memory.

Two cases are excluded from injection. **Module Workers** (created with `type: 'module'`) are passed through unchanged because the `importScripts` mechanism used to load the original script is not available in module scope. **Workers hosted on Google domains** (`.googleapis.com` and `.google.com`) are also excluded because injecting spoofing code into their service workers can break the functionality of Google services such as Maps.

Finally, the injected script patches the `self.location` properties inside the Worker to reflect the original script URL rather than the blob URL. Without this step, any Worker code that reads `location.href` would see an unexpected `blob:` address instead of the real script path, which could cause errors or reveal that the Worker has been modified.

5.6. Same-Origin Iframe Propagation

Every `<iframe>` element creates its own separate JavaScript environment with fresh, unmodified global objects. Fingerprinting scripts exploit this by creating hidden same-origin iframes and reading values like `navigator.userAgent` or `screen.width` from inside them, bypassing all modifications applied to the main page.

To prevent this, the extension intercepts the `contentWindow` and `contentDocument` getters on `HTMLIFrameElement.prototype`. These are the two properties that can be used to access an iframe's internal environment. Whenever a script reads `iframe.contentWindow`, the intercepted getter first applies the spoofing hooks to the iframe's window object before returning it to the caller. This ensures that by the time any code inspects the iframe's environment, the spoofed values are already in place.

Listing 5.12: Iframe `contentWindow` interception

```

1 const iframeCWDestructor = Object.getOwnPropertyDescriptor(
2     HTMLIFrameElement.prototype, 'contentWindow');
3 const origGet = iframeCWDestructor.get;
4
5 Object.defineProperty(HTMLIFrameElement.prototype, 'contentWindow', {
6     get: function() {
7         const cw = origGet.call(this);
8         spoofIframeWindow(cw);

```

```

9      return cw;
10    },
11    configurable: true
12  });

```

The `spoofIframeWindow()` function applies the following modifications to the iframe's environment:

- **Date** and **Intl** are replaced with the already-patched versions from the main page, so all timezone-related functions inside the iframe return the spoofed values.
- The iframe's **navigator** is wrapped in a new **Proxy** using the same spoofed values as the main page. The **Proxy** is constructed using the iframe's own **Proxy** and **Reflect** objects rather than the parent's, because mixing objects from different JavaScript realms would cause type errors.
- The screen property descriptors on the iframe's **Screen.prototype** are redefined to return the chosen resolution, colour depth and pixel ratio.
- The **getParameter** method on the iframe's WebGL rendering context prototypes is replaced to return the spoofed GPU vendor and renderer.

To avoid processing the same iframe twice, a flag (**spoofed**) is set on the iframe's window object after the first patch. On subsequent accesses, the getter checks for this flag and returns the window without modifying it again.

Cross-origin and sandboxed iframes cannot be accessed due to the browser's same-origin security policy. When the getter encounters one of these iframes, the resulting security exception is silently caught and the iframe is left unmodified. This ensures that the extension does not interfere with normal cross-origin content.

5.7. HTTP Header Rewriting

The background service worker (`background.js`) uses the **declarativeNetRequest** API to install a single dynamic rule that rewrites outgoing HTTP request headers for every resource type. The headers modified include **User-Agent**, **Sec-CH-UA**, **Sec-CH-UA-Mobile**, **Sec-CH-UA-Platform** and **Accept-Language**.

The rule is rebuilt on two occasions:

- when the service worker starts up
- whenever a new profile is received from the content script

On startup, before any profile has been received, the service worker uses a hard-coded default profile (Chrome 142). This is necessary because the very first page load sends HTTP headers before the content script has had a chance to generate a session seed and communicate the selected profile. The default ensures that even this initial request carries a spoofed identity rather than exposing the real browser string.

Once the content script selects a profile and sends it to the background script, the rule is rebuilt with the new values and all following requests use the session profile instead of the default.

Listing 5.13: Dynamic rule construction in background.js

```

1  const secChUa = brands
2    .map(b => `"${b.brand}";v="${b.version}"`).join(', ');
3  const acceptLang = langs
4    .map((l, i) => i === 0 ? l : `${l};q=${(1 - i * 0.1).toFixed(1)}')
5    .join(', ');
6
7  chrome.declarativeNetRequest.updateDynamicRules({
8    removeRuleIds: [1],
9    addRules: [{
10      id: 1, priority: 1,
11      action: {
12        type: "modifyHeaders",
13        requestHeaders: [
14          { header: "user-agent", operation: "set", value: FAKE_UA },
15          { header: "sec-ch-ua", operation: "set", value: secChUa },
16          { header: "sec-ch-ua-mobile", operation: "set", value: "?0"
17            },
18          { header: "sec-ch-ua-platform", operation: "set", value:
19            '"Windows"' },
20          { header: "accept-language", operation: "set", value:
21            acceptLang },
22        ]
23      },
24      condition: { urlFilter: "*", resourceTypes: [ "main_frame",
25        "sub_frame", "stylesheet", "script", "image", "font",
26        "xmlhttprequest", "ping", "media", "websocket", "other" ] }
27    }]
28  });

```

Layer Synchronisation

A challenge specific to Manifest V3 is that the content script must run in the **MAIN** world to access and modify the page's JavaScript globals, but scripts running in the **MAIN** world lose access to Chrome extension APIs such as `chrome.runtime.sendMessage`. This means the content script cannot communicate directly with the background service worker.

To bridge this gap, the extension uses a three-file architecture. The content script (`content.js`) selects a profile and posts it to the page using `window.postMessage()`. A separate bridge script (`bridge.js`), which runs in the default isolated extension world, listens for this message and forwards it to the background service worker using `chrome.runtime.sendMessage()`. The background script then saves the profile to `chrome.storage.session` and rebuilds the HTTP header rules to match.

This three-step relay is necessary because Chrome enforces a strict separation between the **MAIN** world (where page scripts and the content script share the same globals) and the isolated extension world (where Chrome APIs are available). The bridge script exists only to cross this boundary.

5.8. Design Decisions and Known Limitations

5.8.1. Design Decisions

Several design choices in the extension reflect trade-offs between coverage, compatibility, and detectability.

Fixed-offset timezone arithmetic was chosen over simply replacing `Intl.DateTimeFormat` with the chosen zone and relying on the browser's own conversion. The browser's internal timezone conversion uses the full IANA rule database, which correctly applies daylight saving time transitions. If the extension had used a different approach for the main thread and the Worker (for example, Intl-based in one and fixed-offset in the other), the two threads would disagree on dates near DST transitions. Using the same fixed-offset arithmetic in both contexts eliminates this class of inconsistency.

Session seeds stored in `sessionStorage` were preferred over per-navigation seeds to maintain profile consistency within a tab. A new random seed on each page load would present a different identity to every page, which would make the extension detectable by any site that reads a fingerprint property before and after a navigation within the same tab.

The blob injection strategy for Workers inlines all profile values as JSON literals rather than communicating them via `postMessage`. This avoids a timing gap during which the worker could read real API values before the spoofing message arrives.

5.8.2. Known Limitations

The following surfaces remain outside the scope of the current implementation:

- **Cross-origin iframes** cannot be patched because the browser enforces the same-origin policy at the JavaScript level. Any access to a cross-origin iframe's `contentWindow` throws a security exception, and the extension catches this silently. A fingerprinting script loaded inside a cross-origin iframe therefore runs in an unpatched environment.
- **Module Workers** (`type: 'module'`) cannot receive the blob injection because module workers do not support `importScripts`. A site using ES module workers could read real browser values from within them.
- **Font enumeration** is not addressed. The canvas font pool randomises the drawing font used by the extension, but it does not prevent scripts from enumerating installed fonts by measuring the pixel width of text rendered in specific typefaces. This technique operates independently of the canvas `fillStyle` and `toDataURL` interceptions.
- **JavaScript API surface fingerprinting** cannot be countered by value substitution alone. Tools that infer the Chrome version from the set of APIs present on `window` (rather than from the UA string value) will still observe the real API surface, which reflects the actual installed Chrome version rather than the spoofed one.

Chapter 6. Testing and Validation

This chapter describes the automated tests developed to validate the anti-fingerprinting extension. Two custom tests were created for this purpose: a multi-user fingerprint test that verifies the correctness and the diversity of every spoofed browser API and an audio waveform test that isolates and visualizes the noise injected into the Web Audio output. Both tests are described below in terms of how they work, the tools used to build them, how they are run and the results obtained.

Beyond these two custom tests, the extension was also checked against the open-source fingerprinting tools FingerprintJS[18], CreepJS[19] and the EFF’s Cover Your Tracks[20]. The FingerprintJS library is integrated directly into the multi-user test and is therefore discussed together with it; the two interactive services are discussed at the end of the chapter.

6.1. Tools and Test Environment

Both tests are written in JavaScript and executed with Node.js. Browser automation is handled by Playwright [21], a framework usually used for testing that can launch and control a real Chromium instance programmatically. Playwright was chosen over a lightweight headless engine because the extension must run in a real browser: only a full Chromium build supports the complete set of Web APIs the extension hooks (Canvas, WebGL, Web Audio, WebRTC, Web Workers), and only a real browser loads unpacked extensions through the `--load-extension` command-line flag.

Both tests use a small local HTTP server, started inside the test with Node’s built-in `http` module. This is needed because the extension’s content scripts only run on pages loaded over `http://` or `https://`; they do not run on `about:blank` or `chrome://` pages. So each test serves a blank page from `127.0.0.1` and opens it before collecting any data, which makes sure the extension is active on the page.

Each simulated user starts with `chromium.launchPersistentContext()` using a new temporary user-data directory. Since the extension creates a unique spoofed profile for each browser session, assigning each instance its own profile directory enables the test to detect a different fingerprint for each simulated user, just as individual real users would have separate profiles. These temporary directories are removed after each session.

6.2. Multi-User Fingerprint Test

6.2.1. Purpose and Structure

FILE: `fp-multiuser-test.js`

The multi-user test launches several independent Chromium instances with the extension loaded each acting as a separate “virtual user” with its own session. For this

test, five users were simulated. For every user it collects a broad fingerprint from the browser and then evaluates a list of assertions against the collected values.

The results are saved both as a JSON file and, through the helper script `fp-report.js`, as an HTML report.

The data collected for each user is grouped into ten categories, corresponding to the browser surfaces the extension modifies:

1. **Navigator** — `userAgent`, `appVersion`, `platform`, `vendor`, `webdriver`, `deviceMemory`, `hardwareConcurrency`, `language`, `languages`, and the User-Agent Client Hints obtained through `navigator.userAgentData.getHighEntropyValues()`.
2. **Screen and Window** — `screen.width/height`, `availWidth/availHeight`, `colorDepth`, `pixelDepth`, `window.outerWidth/outerHeight`, and `devicePixelRatio`.
3. **Timezone** — the offset returned by `getTimezoneOffset()`, the zone and locale from `Intl.DateTimeFormat().resolvedOptions()`, and a set of `Date` operations (string forms, multi-argument and non-ISO constructors, and the `setHours` / `setFullYear` / `setMonth` setters).
4. **Canvas** — the `fillStyle` and `strokeStyle` setters are exercised and read back to verify that the colour perturbation is applied and is deterministic (setting the same input colour twice must yield the same output). The `font` property is also read after assignment to confirm that font substitution takes effect.
5. **WebGL** — the `UNMASKED_VENDOR_WEBGL` and `UNMASKED_RENDERER_WEBGL` strings are queried on both a WebGL1 and a WebGL2 context.
6. **Audio** — an `OfflineAudioContext` with an oscillator feeding a dynamics compressor is rendered and six samples from the middle of the output buffer are stored. The full buffer is then read a second time and compared against the first read to confirm the output is stable within a session.
7. **WebRTC** — an `RTCPeerConnection` (a WebRTC link connecting the local machine with a remote peer) is created and its `iceTransportPolicy` is inspected.
8. **toString() masking** — `.toString()` is called on each overridden function to check whether it still reports `[native code]`.
9. **Web Worker parity** — a `Worker` is created from a `Blob` URL, and it reads the same navigator, timezone and WebGL values internally.
10. **FingerprintJS** — the open-source FingerprintJS library [18] is loaded into the page and its `visitorId`, audio hash, and canvas hash are recorded.

6.2.2. Verification

After collecting the raw values, the test evaluates a set of 55 boolean checks for each user divided into two conceptual categories.

The first category verifies **internal consistency**: that the spoofed values do not contradict one another. For instance:

- the Chrome version extracted from the User-Agent string must match the version reported through the User-Agent Client Hints
- the timezone offset must agree with the offset embedded in `Date.toString()` and `getTimeString()`
- calling `setHours(9)` followed by `getHours()` must return 9
- the WebGL 1 and WebGL 2 contexts must report the same GPU strings
- every value read inside a Web Worker must match its counterpart on the main thread

Consistency is the central concern, because a discrepancy between two related values is itself a strong fingerprinting signal and can betray the presence of spoofing.

The second category verifies **masking**: each overridden function must still return `[native code]` when converted to a string, ensuring the extension remains undetectable by scripts that convert the modified functions to a string format.

Separately from the per-user checks, the report measures **diversity** across users: how many distinct values each attribute takes over the set of virtual users. This metric reflects the anti-tracking objective of producing a different fingerprint for every session.

6.2.3. How It Is Run

The test can be started with one of the following commands:

```
node test
node fp-multiuser-test.js
```

It initially starts a local HTTP server, then sequentially runs each virtual user. For every user, it opens a new Chromium instance with a temporary profile and the extension loaded, navigates to the local page, and performs the collection and check routine within the page context using `page.evaluate()`. Once that returns, it injects the FingerprintJS library with `page.addScriptTag()` and collects its output in a second `page.evaluate()` call. The browser is then closed and the temporary profile directory is deleted. If a user fails, the error is appended to `fp-errors.log` and that user is skipped. When all users have finished, the test writes `fp-results.json` and `fp-results.html` and opens the HTML report in the default browser.

6.2.4. Results

The discussion below is based on a five-user run whose full report is included in the Appendix A.

Correctness

All consistency and masking checks passed for every user. This corresponds to **275** passing checks out of **275** (55 per user), with no failure in any category. In particular, every one of the twelve Worker parity checks passed for every user, confirming

that the spoofed profile is propagated into worker threads and every one of the thirteen `toString()` masking checks passed, confirming that the overridden functions are not distinguishable from native ones by that method.

Diversity

Table 6.1 lists the profile assigned to each of the five users, and Table 6.2 summarises how many distinct values each attribute took across the run.

Table 6.1: Per-user profiles observed

User	Chrome	Timezone	Screen	Mem / CPU	Language
1	138	Europe/Berlin	2560×1440	32 GB / 6	en-GB
2	141	Europe/Warsaw	1366×768	32 GB / 6	fr-FR
3	138	Europe/Warsaw	2560×1440	32 GB / 8	en-GB
4	141	Europe/London	2560×1440	32 GB / 6	es-ES
5	142	Europe/Warsaw	1366×768	32 GB / 8	de-DE

Each user received a distinct combination of attributes, even though individual values repeat. For instance, Users 1 and 3 share Chrome 138, 2560×1440 and **en-GB**, but differ in timezone and CPU count. The important property is not that every individual attribute is unique, but that no two users share the same overall profile. Consequently, as shown in Figure 6.1, all five FingerprintJS `visitorId` hashes are distinct. The `visitorId` represents the fingerprint of each user reported by the open-source tool FingerprintJS.



Figure 6.1: FingerprintJS visitor IDs reported for each of the five virtual users. All five are distinct, confirming that the extension produces a different composite fingerprint per session.

The FingerprintJS audio hash was also distinct for all five users and the canvas hash was distinct for four out of five. Attributes drawn from the profile pools took between two and four distinct values, which is the expected outcome of random selection from a small set of realistic configurations. Coincidental repeats are normal and even desirable, since a shared value places two users in the same anonymity set rather than making either one unique. Memory is a notable example: all five users received 32 GB, resulting in only 1/5 diversity. This is not surprising since the weighted pool assigns approximately a 43% chance to the 32 GB value.

Table 6.2: Diversity of attributes across the five users (distinct values out of 5)

Attribute	Distinct / 5
FingerprintJS <code>visitorId</code>	5
FingerprintJS audio hash	5
FingerprintJS canvas hash	4
WebGL GPU	4
Language	4
Chrome version	3
Timezone	3
Screen resolution	2
CPU cores	2
UTC offset	2
Memory (GB)	1

One detail about how the HTML report presents these numbers: when two users share a hash value, those cells are marked in red as “duplicates.” These are indicators of diversity, not failures in correctness. They simply show a coincidental collision, not a defect in the extension.

6.3. Audio Waveform Test

6.3.1. Purpose

FILE: `audio-waveform-test.js`

The audio waveform test targets the extension’s Web Audio defense specifically. Its goal is to show visually that the noise the extension injects into audio output is large enough to change an audio fingerprint, but small enough to be imperceptible. The approach follows the waveform comparison used by Laperdrix in the FPRandom work [5], which likewise displayed original, modified, and difference waveforms.

6.3.2. How It Is Run

The test requires an audio file named `sample.mp3` in the test folder and is started with:

```
node run test:audio
node audio-waveform-test.js
```

It runs the two captures sequentially, writes `audio-waveform.html`, and opens the report automatically.

6.3.3. Results

Figure 6.2 shows the output for a one-second clip at 44 100 Hz. Plots (a) and (b) are the original and modified waveforms. At normal scale they are visually indistinguishable.

Plot (c) shows the per-sample difference at a magnified vertical scale (on the order of 10^{-6}), revealing the injected noise.

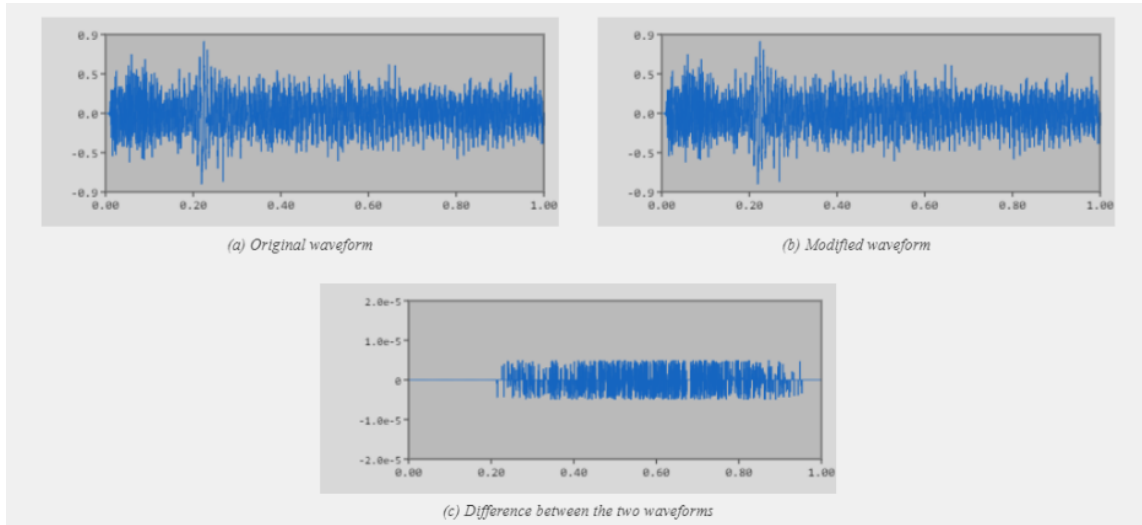


Figure 6.2: Audio waveform comparison: (a) original signal, (b) modified signal with the extension active and (c) the difference between them, magnified to show the injected noise

Out of 44 100 samples, the extension modified 2 038, with a maximum difference of 5.007×10^{-6} . Noise is not added to every sample: the extension chooses a window covering 70–90 % of the buffer and divides it into 200-sample segments. In each chunk, each sample has only a small probability of being modified (at most 25 %, and less near the window edges) and those that are modified receive a random shift of at most $\pm 5 \times 10^{-6}$.

Because most samples are skipped by this probability check, only about 2 000 of samples inside the window were actually changed.

A perturbation of this magnitude is sufficient to alter the hash that fingerprinting scripts compute from the audio buffer, which breaks the audio fingerprint, while remaining far below the threshold of audibility. This test confirms both that noise is injected and its magnitude is small.

6.4. External Validation

In addition to the automated tests, the extension was evaluated against two publicly available fingerprinting services: the Electronic Frontier Foundation’s **Cover Your Tracks** and **CreepJS**. Neither tool was written for this project. Both serve as common reference points in the fingerprinting research field, providing a reliable independent check. The two services have different goals: Cover Your Tracks measures whether a browser can be uniquely identified and whether its fingerprint is randomised, while CreepJS tries to detect spoofing and inconsistencies rather than simply produce an identifier. Together, they show both the strengths of the extension and its limits.

The results reported here come from manual test sessions. Because the extension assigns a new random profile on each browser session, the specific values (User-Agent, timezone, screen size, GPU and so on) differ from one session to another and from one tool to the other. This is expected and is itself a sign that randomisation is working.

6.4.1. Cover Your Tracks

Cover Your Tracks [20] is maintained by the Electronic Frontier Foundation and is the successor to the Panopticlick project discussed in Section 3.1. It reports three high-level indicators:

- whether the browser blocks tracking ads
- whether it blocks invisible trackers
- whether it protects against fingerprinting

Each metric is then listed alongside an estimate of the identifying information it contains, measured in bits.

Overall result

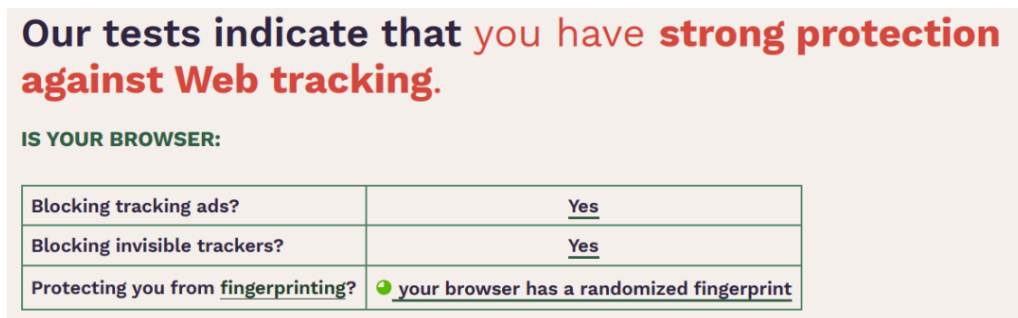


Figure 6.3: Cover Your Tracks overall result: the extension receives a “strong protection against web tracking” conclusion with a randomised fingerprint

As shown in Figure 6.3, Cover Your Tracks concluded that the browser had *strong protection against web tracking*, answering “Yes” to blocking tracking ads, “Yes” to blocking invisible trackers, and reporting that the browser *has a randomised fingerprint*. The summary indicated that the fingerprint was randomised across 335 878 browsers tested in the past 45 days and estimated to carry **at least 18.36 bits** of unique identifying information.

The 18.36-bit value indicates the amount of identifying information a fingerprint would hold if it stayed consistent across sessions. Cover Your Tracks presents this figure with the “randomised” conclusion to show that, while the current fingerprint contains significant detail, its entropy is not useful for tracking. This is because the fingerprint changes with each visit, preventing reidentification of the browser over different sessions.

Characteristics results

The canvas fingerprint hash, the WebGL fingerprint hash, the AudioContext fingerprint and the hardware concurrency value were all reported as *randomised by first party domain* rather than as stable identifiers. These correspond precisely to the attributes the extension is designed to protect and their classification as randomised constitutes direct external confirmation that the canvas, WebGL, audio, and hardware concurrency defences are functioning as intended.

The remaining metrics were documented as stable values: the User-Agent (11.77 bits), the HTTP_ACCEPT headers (17.36 bits), the timezone and offset, the screen size and colour depth, the system font list, the WebGL vendor and renderer string, the platform,

the language and the device memory. This result is anticipated, as the extension does not alter these attributes between page loads. It assigns a fixed, plausible profile that remains consistent throughout the session. Notably, the WebGL vendor/renderer string (reported in this session as an NVIDIA GeForce RTX 3060 via ANGLE, contributing 12.83 bits) was among the items with the highest entropy in the report. This aligns with the intended design: the extension replaces a realistic GPU string instead of hiding it, making the spoofed value appear real to the fingerprinting tool.

6.4.2. CreepJS

CreepJS [19] is a testing and demonstration tool that is much more assertive than Cover Your Tracks. Instead of just generating an identifier, it actively detects *lies* (instances where one API reports a value that conflicts with a value derived from another method) as well as indicators of automation, headless browsers and proxy-driven property modifications.

Passed Checks

Several features of the extension resisted testing by CreepJS. The dedicated **iframe** test passed: across all of the iframe contexts CreepJS exercises (**contentWindow**, nested frames, same-origin frames, and so on), the reported User-Agent, the reconstructed User-Agent, the Chrome version (reported as Chrome 142 in that session), the platform **Win32**), and the canvas hash were identical in every context and CreepJS marked the test as passed.

This confirms that the spoofed profile is applied uniformly to child frames and does not fall back to the real values inside them.

Likewise, in the workers comparison the main window, the dedicated worker and the shared worker all reported the same user identity, showing that the profile is propagated consistently into those worker types.

Detected Issues

CreepJS also detected the extension in several ways.

On the timezone page, it marked the reported timezone (**Europe/Warsaw**) and offset (-120) as **fake**, having computed a different location internally (**Europe/Simferopol**). CreepJS can tell that the timezone has been overridden, even though the offset and the string representations are internally consistent with each other. Several other timezone checks did pass, including the date string format, the zone name, the epoch time across four methods and the UTC ISO time.

On the screen page, CreepJS flagged the spoofed screen dimensions (2560×1440) because they did not match the actual viewport dimensions measured through CSS media queries and DOM methods. The extension sets **screen.width** and **screen.height** to the spoofed values, but the real browser window remains at its actual size, so CSS-based measurements such as **@media** queries and **visualViewport** report the true dimensions. This mismatch is what CreepJS detects.

On the main page, under the “Headless” panel, CreepJS reported three scores shown in Figure 6.4.

These scores reflect how closely the browser’s behaviour matches patterns typical of automated or modified browsers. The “like headless” score measures how many traits

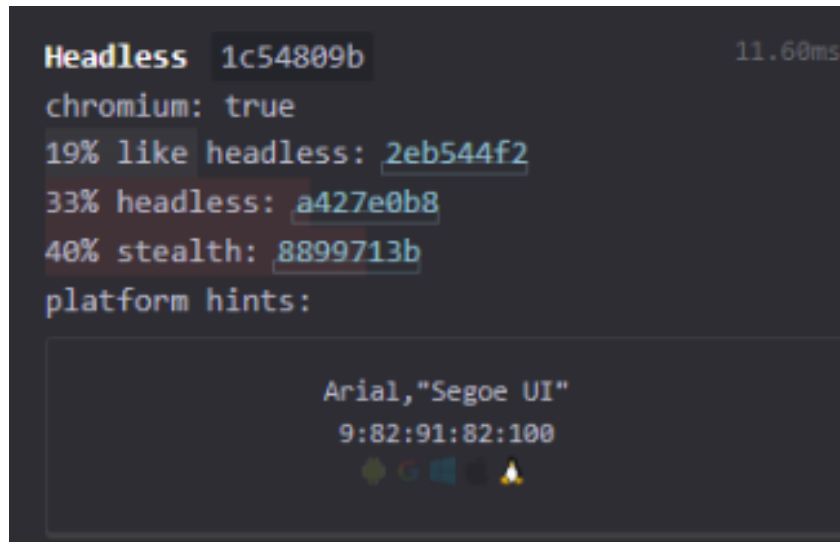


Figure 6.4: Headless Section

resemble a headless browser environment. The “headless” score indicates a stronger match against known headless browser signatures. The “stealth” score reflects how many signs of anti-detection tools CreepJS found. In other words, how much the browser appears to be actively hiding its true identity. A higher stealth score means more indicators of spoofing were detected.

These scores confirm that CreepJS can recognise the browser as modified, even though it cannot fully identify which specific values are spoofed.

Service Worker Inconsistency

One limitation to note is in the worker comparison. While the dedicated and shared workers aligned with the main window, the *service worker* reported a different profile: a distinct User-Agent hash, a different GPU (AMD instead of NVIDIA), a different hardware setup, a different timezone (Europe/Bucharest instead of Europe/Warsaw), and a different language (en-GB instead of fr-FR). CreepJS flagged all these mismatched values.

This shows that the extension did not apply the same spoofed profile to the service-worker context as it did to the main window and the other worker types.

A tracker that can read values from both the window and a service worker could notice this difference, making it a real inconsistency and a candidate for future improvement.

Chapter 7. User's Manual

7.1. System Requirements

7.1.1. Hardware

The extension runs in the browser and does not require any specific hardware beyond what is needed to run a modern web browser.

7.1.2. Software

- **Operating system:** Windows 10 or later, macOS 10.15 or later, or any Linux distribution that supports Chromium-based browsers.
- **Browser:** Google Chrome (version 120 or later recommended) or any Chromium-based browser that supports Manifest V3 extensions.
- **No additional software** is required. The extension does not depend on any external tools, servers or runtime environments.

7.2. Installation

The extension is not published on the Chrome Web Store and must be installed manually in developer mode. Follow these steps to successfully add the extension:

1. Download the extension locally

Download or clone the extension folder to a location on your computer. The folder should contain at least the following files: `manifest.json`, `content.js`, `background.js` and `bridge.js`.

2. Open the extensions page - Figure 7.1

In browser, type `chrome://extensions` in the address bar and press Enter.

3. Enable Developer Mode - Figure 7.2

In the top-right corner of the extensions page, toggle the “Developer mode” switch to the on position.

4. Load the extension - Figure 7.3

Click the “Load unpacked” button that appears after enabling Developer Mode. A file browser dialog will open. Navigate to the extension folder and select it.

5. Verify the installation - Figure 7.4

The extension should now appear in the list on the extensions page with its name (“Anti-Fingerprint Extension”) and a toggle switch to enable or disable it. Make

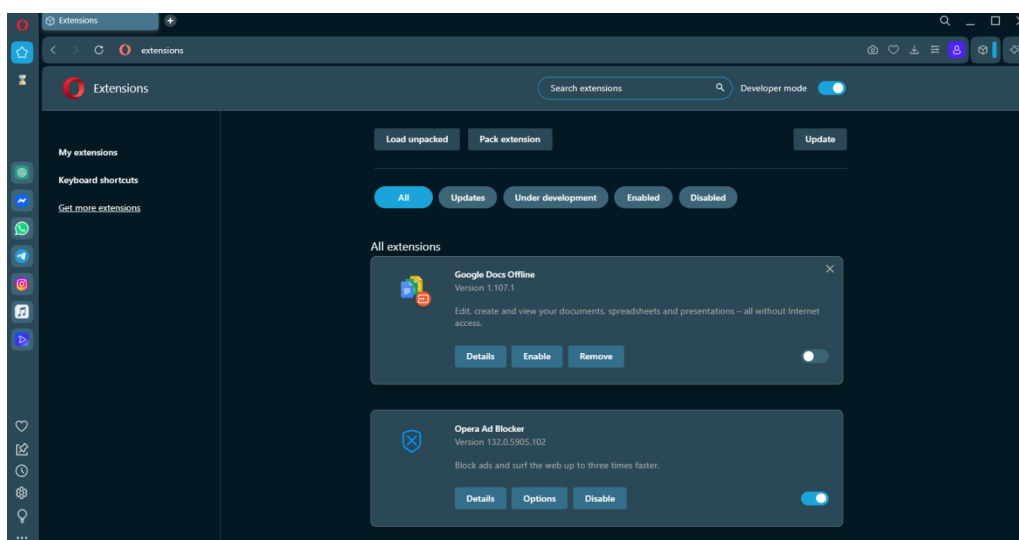


Figure 7.1: The Chrome extensions page

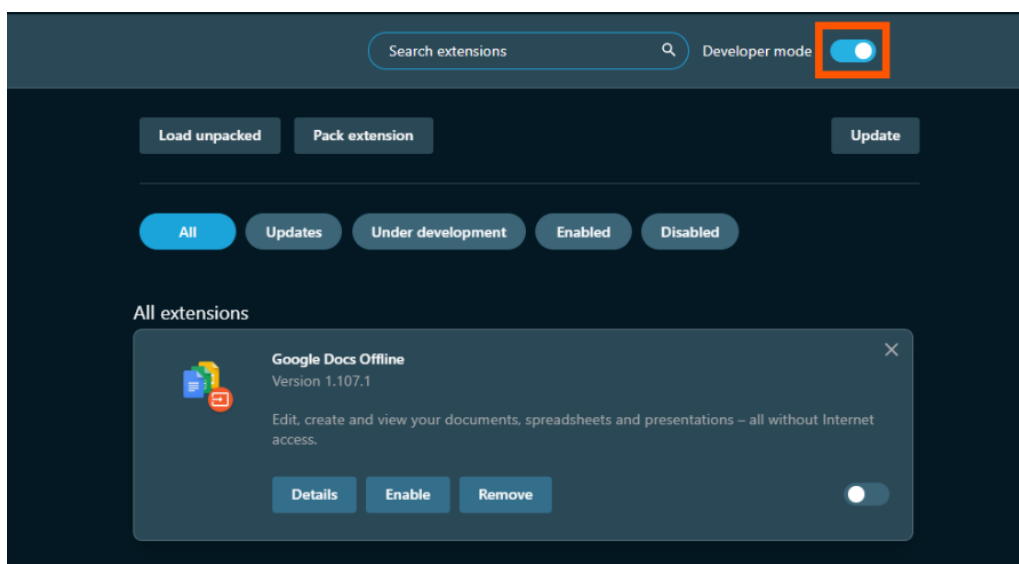


Figure 7.2: Enabling Developer Mode

sure the toggle is set to on.

The extension is now active and will apply to all websites visited in this browser. No restart is required.

7.3. Uninstallation

To remove the extension, go to `chrome://extensions`, find the “Anti-Fingerprint Extension” card and click the “Remove” button.

Alternatively, the extension can be temporarily disabled by toggling its switch to the off position without removing it.

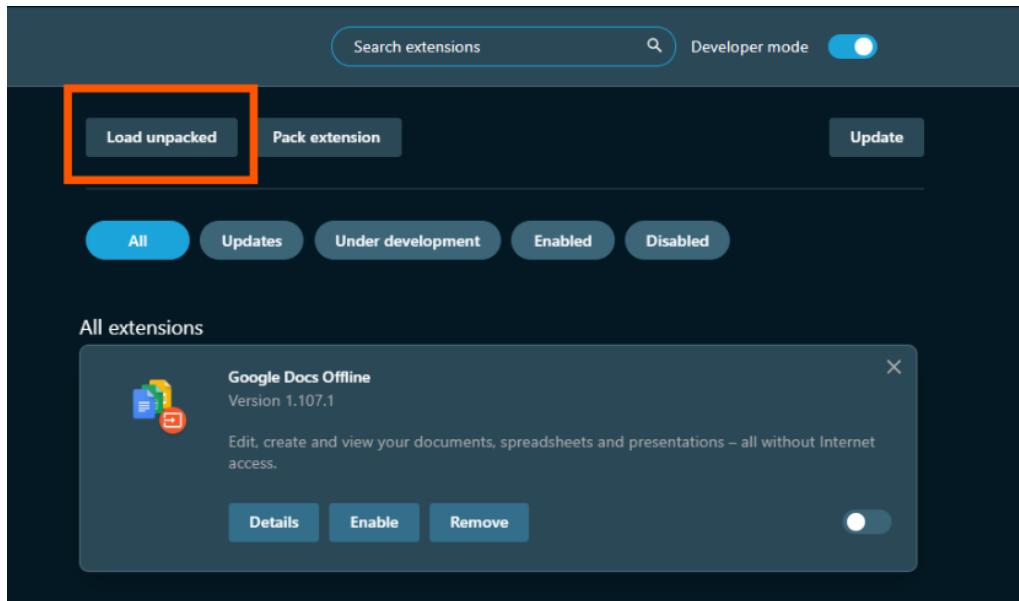


Figure 7.3: Clicking “Load unpacked” and selecting the extension folder

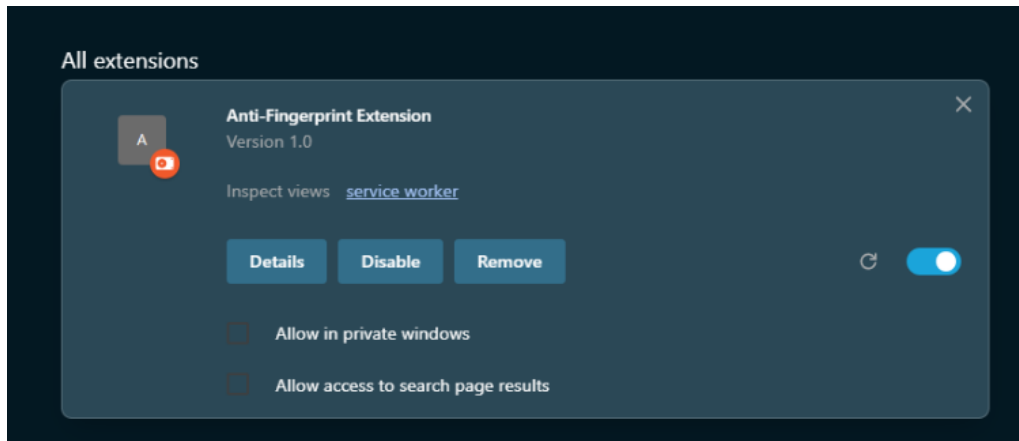


Figure 7.4: The extension successfully loaded.

7.4. Usage

7.4.1. Everyday Use

The extension works automatically and requires no interaction from the user. Once installed, it runs silently in the background on every page the browser loads. There is no popup, no toolbar icon to click, and no settings to configure.

Each time the browser is opened (each new session), the extension randomly selects a new identity profile. This profile includes a User-Agent string, a screen resolution, a timezone, a language, a GPU name, the number of CPU cores, and the amount of device memory. All of these values are drawn from pools of common, realistic configurations so that the browser blends in with ordinary traffic.

The profile stays the same for the entire session, meaning that all pages visited during that session see the same consistent identity. When the browser is closed and reopened, a new profile is selected.

7.4.2. What Changes

From the user's perspective, most browsing will feel completely normal. The following differences may occasionally be noticeable:

- **Language:** The browser may report a different preferred language (for example, French or German instead of the user's real language). Some websites may display content in that language. If this happens, the user can change the language using the website's own language selector.
- **Timezone:** The browser reports a spoofed timezone from a European timezone pool. Websites that display the local time based on the browser's timezone may show a different time. The system clock is not affected.

7.4.3. Verifying That the Extension Works

To confirm that the extension is active and applying a different profile, the user can visit a fingerprinting test website such as:

- Cover Your Tracks: <https://coveryourtracks.eff.org/>
- CreepJS: <https://abrahamjuliot.github.io/creepjs/>

Chapter 8. Conclusions

8.1. Summary of Contributions

This work presented a Manifest V3 Chrome extension that defends against browser fingerprinting by replacing identifiable browser attributes with values drawn from pools of realistic configurations sourced from real-world usage statistics. The extension operates entirely on the client side, requires no external server or additional software and needs no user interaction after installation.

The implementation addresses eleven fingerprinting vectors through a two-layer architecture: a background service worker rewrites outgoing HTTP headers (User-Agent, Sec-CH-UA, Sec-CH-UA-Platform, Sec-CH-UA-Mobile, Accept-Language) using the declarativeNetRequest API, while a content script intercepts JavaScript APIs covering the navigator properties, screen dimensions, the full Date and Intl.DateTimeFormat surface, canvas rendering, WebGL vendor and renderer strings, audio buffer data, RTCPeerConnection and the speech synthesis voice list. The background script, content script, and bridge script coordinate through `chrome.storage.session` to keep both layers in sync. All values are derived from a single session seed through a Mulberry32 PRNG, ensuring consistency within a session and diversity across sessions. The profile is propagated into dedicated workers, shared workers, and same-origin iframes, and all overridden functions are masked through a `Function.prototype.toString` hook.

8.2. Analysis of Results

The multi-user test launched five independent browser sessions and verified 55 checks per user (275 total), all passing. These covered navigator consistency, screen and window properties, timezone arithmetic (constructor, getters, setters, string formatting), canvas determinism, WebGL parity across both context versions, audio read stability, twelve worker parity checks, and thirteen `toString` masking checks. The FingerprintJS library produced a distinct visitor ID for each session.

The audio waveform test confirmed that the injected noise has a maximum magnitude of 5.007×10^{-6} — too small to be audible, but sufficient to alter the audio fingerprint hash.

Cover Your Tracks reported the browser as having *strong protection against web tracking* with a *randomised fingerprint*, classifying the canvas, WebGL, audio, and hardware concurrency attributes as randomised. CreepJS confirmed consistent spoofing across all iframe contexts and across dedicated and shared workers.

However, the evaluation also exposed weaknesses. CreepJS detected the spoofed timezone through an independent computation method, and flagged the spoofed screen dimensions because CSS media queries and the `visualViewport` API still report the real window size. It assigned a stealth score of 40 %, indicating signs of active modification.

Most importantly, the service worker received a different profile from the main window (different User-Agent, GPU, timezone, and language), because the blob injection strategy does not apply to service workers.

8.3. Possibilities for Improvement

Service worker propagation

The service worker inconsistency is the most significant gap. The current blob injection wraps worker scripts with `importScripts()`, which does not apply to service workers. Extending the spoofing to this context would close the most important inconsistency found during testing.

Screen and timezone detection

Intercepting CSS media queries and the `visualViewport` API to return values consistent with the spoofed screen size would remove the screen mismatch that CreepJS detected. Similarly, covering the additional timezone derivation methods that CreepJS uses would strengthen the timezone defence.

Larger value pools

In the five-user test, all users received 32 GB of memory (diversity of 1/5) and screen resolution had a diversity of only 2/5. Expanding the pools and refining the weights would reduce such coincidental collisions.

Uncovered vectors

Cross-origin iframes cannot be patched due to the same-origin policy. Module workers cannot receive the blob injection because they do not support `importScripts`. Font enumeration through pixel-width measurement and JavaScript API surface fingerprinting (inferring the Chrome version from the set of APIs on `window`) remain outside the current scope.

User interface

Adding a popup or options page would allow users to see the active profile, force a refresh or exclude specific websites from spoofing.

Bibliography

- [1] P. Eckersley, “How unique is your web browser?” in *Proceedings of the 10th International Conference on Privacy Enhancing Technologies (PETS)*, 2010.
- [2] S. Englehardt and A. Narayanan, “Online tracking: A 1-million-site measurement and analysis,” in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2016.
- [3] Electronic Frontier Foundation, “Privacy badger,” <https://privacybadger.org/>.
- [4] R. Hill, “uBlock Origin,” <https://github.com/gorhill/uBlock>.
- [5] P. Laperdrix, B. Baudry, and V. Mishra, “FPRandom: Randomizing core browser objects to break advanced device fingerprinting techniques,” in *9th International Symposium on Engineering Secure Software and Systems (ESSoS)*, Bonn, Germany, 2017.
- [6] The Tor Project, “Tor browser design document,” <https://2019.www.torproject.org/projects/torbrowser/design/>, 2023.
- [7] Brave Software, “Brave browser privacy features,” <https://brave.com/privacy-features/>, 2023, [Accessed 2026].
- [8] StatCounter, “Global stats – browser and screen resolution market share,” <https://gs.statcounter.com>.
- [9] Valve Corporation, “Steam hardware and software survey,” <https://store.steampowered.com/hwsurvey>.
- [10] AmIUnique, “Learn how identifiable you are on the internet,” <https://amiunique.org>.
- [11] P. Laperdrix, W. Rudametkin, and B. Baudry, “Mitigating browser fingerprint tracking: Multi-level reconfiguration and diversification,” in *10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2015)*, 2015.
- [12] N. Nikiforakis, W. Joosen, and B. Livshits, “PriVaricator: Deceiving fingerprinters with little white lies,” in *Proceedings of the 24th International Conference on World Wide Web (WWW ’15)*. International World Wide Web Conferences Steering Committee, 2015.
- [13] N. Nikiforakis, A. Kapravelos, W. Joosen, C. Kruegel, F. Piessens, and G. Vigna, “Cookieless monster: Exploring the ecosystem of web-based device fingerprinting,” in *Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP ’13)*. IEEE Computer Society, 2013.

- [14] K. Mowery and H. Shacham, “Pixel perfect: Fingerprinting canvas in HTML5,” in *Proceedings of W2SP 2012*. IEEE Computer Society, 2012.
- [15] A. Gómez-Boix, P. Laperdrix, and B. Baudry, “Hiding in the crowd: An analysis of the effectiveness of browser fingerprinting at large scale,” in *Proceedings of the World Wide Web Conference (WWW)*, 2018.
- [16] P. Laperdrix, W. Rudametkin, and B. Baudry, “Beauty and the beast: Diverting modern web browsers to build unique browser fingerprints,” in *37th IEEE Symposium on Security and Privacy (S&P 2016)*, 2016.
- [17] Wikipedia, “ISO 8601 — date and time format,” https://en.wikipedia.org/wiki/ISO_8601.
- [18] FingerprintJS Inc., “FingerprintJS – browser fingerprinting library,” <https://github.com/fingerprintjs/fingerprintjs>, 2024.
- [19] A. Juliot, “CreepJS – creep with JavaScript,” <https://abrahamjuliot.github.io/creepjs/>, 2021, [Accessed 2026].
- [20] Electronic Frontier Foundation, “Cover your tracks,” <https://coveryourtracks.eff.org/>, 2020.
- [21] Microsoft, “Playwright – end-to-end testing framework,” <https://playwright.dev/>, 2024.

Appendix A. Multi-Users Test Results

Anti-Fingerprint - Test Report

05/07/2026, 20:46:53 | 5 users

FINGERPRINTJS VISITOR ID (FINGERPRINT)

All 5 sessions produced a distinct fingerprint.

USER 1 38d795d91771adc133d0f97837b7502f	USER 2 ca97c800c109d6cc3b9421ec798f4a10	USER 3 bcdeb986913ae9658700d8c5b2293b14
USER 4 8050266cb65f89d40660570e8cd7c15a	USER 5 5409f6870364f01a19967db8053a5729	

FINGERPRINT PROFILES

User	Chrome	Platform	Vendor	Timezone	UTC offset	Screen	Avail	colorDepth / DPR	Memory	CPU	Language	V
User 1	138	Win32	Google Inc.	Europe/Berlin	+2h	2560x1440	2560x1400	24 / 1	32 GB	6	en-GB	A
User 2	141	Win32	Google Inc.	Europe/Warsaw	+2h	1366x768	1366x728	24 / 1	32 GB	6	fr-FR	A
User 3	138	Win32	Google Inc.	Europe/Warsaw	+2h	2560x1440	2560x1400	24 / 1	32 GB	8	en-GB	A
User 4	141	Win32	Google Inc.	Europe/London	+1h	2560x1440	2560x1400	24 / 1	32 GB	6	es-ES	A
User 5	142	Win32	Google Inc.	Europe/Warsaw	+2h	1366x768	1366x728	24 / 1	32 GB	8	de-DE	A

CANVAS

Check	User 1		User 2		User 3		User 4		User 5	
font after hook	—	14px Cambria	—	14px "Lucida Console"	—	14px "Times New Roman"	—	14px "Comic Sans MS"	—	14px Verdana
fillStyle consistent		#646464 / #646464		#646464 / #646464		#646464 / #646464		#646464 / #646464		#646464 / #646464
strokeStyle is valid rgb		#c86432		#c86432		#c86432		#c86432		#c86432

FINGERPRINTJS COMPONENTS

User	visitorId	Audio hash		Audio duration	Canvas hash (geometry)	
User 1	38d795d91771adc133d0f97837b7502f		124.04347039503045	4.0 ms		data:image/png;base64,iVBORw0KGgoAAAANSUhEUgA
User 2	ca97c800c109d6cc3b9421ec798f4a10		124.04348493204452	3.0 ms		data:image/png;base64,iVBORw0KGgoAAAANSUhEUgA
User 3	bcdeb986913ae9658700d8c5b2293b14		124.04347020876594	4.0 ms		data:image/png;base64,iVBORw0KGgoAAAANSUhEUgA
User 4	8050266cb65f89d40660570e8cd7c15a		124.04347527516074	3.0 ms		data:image/png;base64,iVBORw0KGgoAAAANSUhEUgA
User 5	5409f6870364f01a19967db8053a5729		124.04347986471839	3.0 ms		data:image/png;base64,iVBORw0KGgoAAAANSUhEUgA

WEBRTC - ICE POLICY

User	iceTransportPolicy (expected: relay)
User 1	relay
User 2	relay
User 3	relay
User 4	relay
User 5	relay

NAVIGATOR

Check	User 1		User 2		User 3		User 4		User 5	
vendor = Google Inc.		Google Inc.		Google Inc.		Google Inc.		Google Inc.		Google Inc.
webdriver = false		false		false		false		false		false
UA <-> UAData version		UA:138 UAD:138		UA:141 UAD:141		UA:138 UAD:138		UA:141 UAD:141		UA:142 UAD:142
UAData platform=Windows		Windows		Windows		Windows		Windows		Windows
UAData arch=x86		x86		x86		x86		x86		x86
UAData bitness=64		64		64		64		64		64

SCREEN & WINDOW

Check	User 1	User 2	User 3	User 4	User 5
avail <= full	2560x1400 / 2560x1440	1366x728 / 1366x768	2560x1400 / 2560x1440	2560x1400 / 2560x1440	1366x728 / 1366x768
colorDepth = 24	24	24	24	24	24
pixelDepth = 24	24	24	24	24	24
devicePixelRatio = 1	1	1	1	1	1
outerWidth = screen.width	2560 / 2560	1366 / 1366	2560 / 2560	2560 / 2560	1366 / 1366
outerHeight = screen.height	1440 / 1440	768 / 768	1440 / 1440	1440 / 1440	768 / 768

TIMEZONE (DATE / INTL)

Check	User 1	User 2	User 3	User 4	User 5
offset <-> Date.toString()	GMT+0200	GMT+0200	GMT+0200	GMT+0100	GMT+0200
offset <-> toTimeString()	GMT+0200	GMT+0200	GMT+0200	GMT+0100	GMT+0200
getHours() correct	12	12	12	12	12
constructor multi-arg getHours	10	10	10	10	10
constructor non-ISO string	12	12	12	12	12
setHours/getHours	9	9	9	9	9
setFullYear/getFullYear	2026	2026	2026	2026	2026
setMonth/getMonth	3	3	3	3	3

WEBGL

Check	User 1	User 2	User 3	User 4	User 5
WebGL1 vendor spoofed	Google Inc. (Intel)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)
WebGL1 renderer spoofed	ANGLE (Intel, Intel(R) Iris(R) Xe Graphics Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 4060 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce GTX 1650 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)
WebGL2 vendor spoofed	Google Inc. (Intel)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)
WebGL2 renderer spoofed	ANGLE (Intel, Intel(R) Iris(R) Xe Graphics Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 4060 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce GTX 1650 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)
WebGL1 = WebGL2	Google Inc. (Intel)/ANGLE (Intel, Intel(R) Iris(R) Xe Graphics Direct3D11 vs_5_0 ps_5_0, D3D11)	Google Inc. (NVIDIA)/ANGLE (NVIDIA, NVIDIA GeForce RTX 4060 Direct3D11 vs_5_0 ps_5_0, D3D11)	Google Inc. (NVIDIA)/ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)	Google Inc. (NVIDIA)/ANGLE (NVIDIA, NVIDIA GeForce GTX 1650 Direct3D11 vs_5_0 ps_5_0, D3D11)	Google Inc. (NVIDIA)/ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)

AUDIO

Check	User 1	User 2	User 3	User 4	User 5
no error	ok	ok	ok	ok	ok
repeated read consistent	true	true	true	true	true

WORKER PARITY

Check	User 1	User 2	User 3	User 4	User 5
UA match	138	141	138	141	142
platform match	Win32	Win32	Win32	Win32	Win32
memory match	32	32	32	32	32
CPU match	6	6	8	6	8
language match	en-GB	fr-FR	en-GB	es-ES	de-DE
TZ offset match	-120	-120	-120	-60	-120
TZ name match	Europe/Berlin	Europe/Warsaw	Europe/Warsaw	Europe/London	Europe/Warsaw

Check	User 1	User 2	User 3	User 4	User 5
Date.toString() match	Sun Jul 05 2026 12:00:00 GMT+0200 (Central European Summer Time)	Sun Jul 05 2026 12:00:00 GMT+0200 (Central European Summer Time)	Sun Jul 05 2026 12:00:00 GMT+0200 (Central European Summer Time)	Sun Jul 05 2026 12:00:00 GMT+0100 (British Summer Time)	Sun Jul 05 2026 12:00:00 GMT+0200 (Central European Summer Time)
constructor multi-arg match	10	10	10	10	10
setHours match	9	9	9	9	9
WebGL vendor match	Google Inc. (Intel)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)	Google Inc. (NVIDIA)
WebGL renderer match	ANGLE (Intel, Intel(R) Iris(R) Xe Graphics Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 4060 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce GTX 1650 Direct3D11 vs_5_0 ps_5_0, D3D11)	ANGLE (NVIDIA, NVIDIA GeForce RTX 3060 Direct3D11 vs_5_0 ps_5_0, D3D11)

TOSTRING() MASKING

Check	User 1	User 2	User 3	User 4	User 5
Date.prototype.getTimezoneOffset					
Date.prototype.getHours					
Date.prototype.toString					
Date.prototype.toTimeString					
Date.prototype.toLocaleString					
Date constructor					
Intl.DateTimeFormat					
Intl.DateTimeFormat.resolvedOptions					
WebGLRenderingContext.getParameter					
WebGL2RenderingContext.getParameter					
AudioBuffer.getChannelData					
RTCPeerConnection					
Worker constructor					

DIVERSITY

Property	Distinct values	Spread across 5 users
Chrome version	3 / 5	
Timezone	3 / 5	
TZ offset	2 / 5	
Screen	2 / 5	
Memory (GB)	1 / 5	
CPU cores	2 / 5	
Language	4 / 5	
WebGL GPU	4 / 5	
FPJS visitorId	5 / 5	
FPJS audio hash	5 / 5	
FPJS canvas hash	4 / 5	

Appendix B. Published Papers

Article Title: *Profile-Based Privacy-Preserving Extension for Internet Browsers*

This paper was presented during the Computer Science Student Conference (CSSC) 2026, and managed to receive **3rd Place** and **a special prize from the Bosch company**.

Profile-Based Privacy-Preserving Extension for Internet Browsers

Rîsteiu Ioana-Stefania
 Technical University of Cluj-Napoca,
 Faculty of Automation and Computer
 Science
 Cluj-Napoca, Romania
ioana.stefania.risteiu@gmail.com

Prof. Coordinator Adrian Colesa
 Department of Computer Science,
 Technical University of Cluj-Napoca,
 Romania
adrian.colesa@cs.utcluj.ro

Prof. Coordinator István-Attila Császár
 Department of Computer Science,
 Technical University of Cluj-Napoca,
 Romania
istvan.csaszar@cs.utcluj.ro

Abstract—Browser fingerprinting enables websites to identify and track users across sessions without cookies by combining device and software properties into a unique identifier. In this paper, we present an anti-fingerprinting browser extension for Chromium browsers which replaces user's unique browser fingerprint with one that can be considered part of a group of artificially generated profiles. Rather than blocking fingerprinting APIs, which can be easily detected, the extension spoofs browser properties such as Navigator, Screen, Canvas, WebGL, Audio, Timezone, WebRTC, and Web Worker execution contexts. A seeded pseudo-random number generator ensures internal consistency within a session. The extension was evaluated by comparing fingerprint values across different users, demonstrating how each property changes between sessions.

I. INTRODUCTION

Online tracking is constantly evolving beyond the era of third-party cookies. As browsers restrict or block persistent identifiers, advertisers and analytics providers have shifted their focus to tracking techniques that do not require storage on the user's device. Browser fingerprinting is the most widespread technique. It works by making multiple API calls to gather browser properties, such as screen dimensions, installed fonts, GPU renderer strings, and audio processing characteristics. After reading all the properties, a server can construct a near-unique identifier from how a particular browser and device combination renders content [1].

Unlike cookies, fingerprints cannot be deleted. They persist across private browsing sessions, often survive browser updates, and do not require user action to gather. Research indicates that even a small set of properties can uniquely identify many browser users [1,2].

Existing defences follow one of two strategies. The first, blocking, involves entirely preventing access to the fingerprinting API, such as returning null, throwing exceptions, or providing a fixed constant. This approach can be identified because the absence of a value, or a synthetic one, acts as an indicator that a privacy tool is active [3]. The second strategy is noise injection. This approach returns slightly different values each time a request is made, making consistent fingerprinting impossible. This method interferes with internal consistency, a site that requests the same property twice and receives different values each time can be easily flagged as trying to use anti-fingerprinting techniques.

This paper presents an alternative to these methods, profile pooling. Instead of blocking or randomising property values per request, the extension assigns each browser session a coherent fake identity selected from a pool of realistic profiles. Within a session, every API returns the same values, so the user identifier remains consistent throughout the active session. Across sessions, the profile changes, so two visits to the same website cannot be linked. More importantly, because profiles are drawn from real-world value distributions, individual users can easily blend into the already detected fingerprints rather than stand out as privacy-tool users.

The implementation takes the form of a Chrome Manifest V3 extension organised into two layers that work together. The first handles HTTP request headers, rewriting User-Agent, Sec-CH-UA, and Accept-Language through the *declarativeNetRequest* API. The second injects a content script at document start, intercepting JavaScript APIs before any page code executes. To cover Web Worker contexts, the extension uses a Blob URL mechanism to inject the same profile into Workers, so fingerprinting scripts running there cannot observe any inconsistencies with the main page.

Section 6 covers the evaluation methods. Among the approaches used, we implemented a custom testing framework that simulates five separate users, each starting a fresh browser session, and checks whether the spoofed values change across sessions as expected.

II. BACKGROUND AND RELATED WORK

A. Browser Fingerprinting Mechanisms

Eckersley's 2010 research [1] showed that browser fingerprinting is highly feasible. By combining data such as the User-Agent string, screen resolution, system fonts, timezone and plugin list, it was possible to uniquely identify 83.6% of browsers in a dataset of over 470000 visitors. The findings indicate that each property adds entropy bits, and the combined entropy of multiple properties typically exceeds 33 bits, which is sufficient to distinguish a single user from the global internet population.

Englehardt and Narayanan's 2016 study [2] analysed fingerprinting practices on a large scale, discovering that out of the top 100000 visited websites, 14371 are running canvas fingerprinting scripts. This technique involves rendering text

and shapes off-screen and hashing the pixel data. Variations in GPU rendering pipelines, anti-aliasing, and subpixel rendering lead to differences between devices, making the resulting hash a hardware-dependent identifier that remains consistent over time and is resistant to most user interventions.

A later study by Gomez-Boix [5] collected over 2 million fingerprints from a general news website and found that only 33.6% were unique, much lower than earlier results. This difference is likely because previous studies collected data from privacy-focused websites, where visitors tend to have more unusual browser setups. The finding shows that fingerprint uniqueness depends on the type of audience being measured.

Modern fingerprinting scripts have advanced beyond these techniques. WebGL graphics hardware descriptors expose the GPU model and driver version. The Web Audio APIs reveal numerical properties of the Digital Signal Processing (DSP) pipeline. Timezone APIs reveal geographic location. User Agent Data exposes the UA structure, including architecture, platform version and device model. Any of these may be combined to construct a fingerprint with high precision.

B. Existing Defences

Tor Browser [4] takes the most aggressive blocking approach, built around a single core principle: uniformity. Rather than spoofing or randomising values, Tor Browser offers a fixed, standardised configuration so that every user presents an identical fingerprint. If all users look the same, fingerprinting becomes useless, and no individual can be identified.

In practice, this means imposing a series of constraints. Some attributes, such as the operating system and language, are essential for basic functionality and cannot be fully hidden or spoofed. Rather than removing them, Tor Browser reduces variation in these attributes to make individual users less distinguishable. Font enumeration is limited, character fallback is applied, screen and window dimensions are standardised through letterboxing, and the set of accepted languages is restricted to a small, predefined list. However, it comes with significant costs. The fixed window size and disabled APIs cause usability problems on many websites, and the overall browsing experience is constrained.

Brave Browser [7] takes a softer version of the same approach, applying per-session randomization to canvas and audio outputs while keeping other properties close to real values. Unlike Tor Browser, Brave does not enforce a uniform identity across users, so the anonymity set is not well-defined.

Privacy Badger, uBlock Origin, and similar browser extensions block HTTP requests to domains known to host fingerprinting scripts, using blocklists such as EasyPrivacy or Disconnect.me. When a page attempts to load a third-party analytics or tracking script from a known domain, the extension intercepts the network request and prevents it from executing.

C. Our approach

Our approach is built on three properties. First, session consistency is achieved by using the same PRNG sequence, seeded at the start of the session, to select the spoofed value returned within a single browser session. Second, cross-session diversity is achieved because the seed is stored in “sessionStorage” and discarded when the tab closes, resulting in each new session having a different profile. A fingerprinting service that records a visitor's fingerprint cannot match it to a future visit. The third is profile credibility. The spoofed values are not arbitrary constants, they are drawn from pools of values observed in real browsers, covering common Chrome versions, GPU configurations, screen resolutions, and language settings. A user running the extension does not stand out as a privacy tool user.

III. DESIGN

A. Threat Model

We consider a tracking server that reads browser properties using standard JavaScript APIs, may call the same API multiple times within a session to test for inconsistency and may compare values across the HTTP header layer and the JavaScript layer to detect mismatches. Additionally, the server may inspect overridden functions using “toString()” function to detect extension modifications and run fingerprinting code inside Web Workers to get around content script injection.

B. Architecture

- **Layer 1 - HTTP Header Rewriting**

The background service worker manages HTTP-level spoofing. When the extension is activated in the browser, the service worker sets up a rewriting rule that intercepts all outgoing HTTP requests and replaces the headers with a chosen fake profile. These headers include information that the browser automatically sends with each request before the page loads: the browser name and version, structured identity fields encoding the same information in a machine-readable format, a flag indicating whether the device is mobile or desktop, the operating system name and the user's preferred language list. Together, these headers create a consistent identity that the server can read without executing any JavaScript. The extension rewrites all of these headers together because fingerprinting services check if there are mismatches between the header values. For example, the version numbers of each property must remain compatible with those of the others.

- **Layer 2 - JavaScript API Hooking**

The second component runs directly within the page and intercepts JavaScript API calls before any website code executes. It is injected at the earliest stage of page loading, before the browser parses or runs any scripts. This timing is essential, as fingerprinting scripts loading early could otherwise access real browser property values before protection is active. Once injected, the spoofing code installs modified browser APIs so that all succeeding scripts,

whether from the website or third-party trackers, only access the spoofed values.

C. Session Seed and Pseudo-Random number generator (PRNG)

Session consistency is achieved by storing a seed in a storage area associated with the current browser tab. During the first time the page runs in a tab, a short random string is generated using the browser's native randomness source, combining two generated numbers for unpredictability. The seed is reused across navigations within the same tab and lost when the tab closes, so the next session receives a different profile.

The seed is obtained using the Mulberry32 generator. Two 32-bit state values are first derived from it using polynomial rolling hashes with different multipliers:

$$a_0, b_0 = \sum_{i=1}^n c_i \cdot m^{n-i} \bmod 2^{32}, \quad m \in \{31, 37\} \quad (1)$$

On each call, both states advance by fixed odd increments, wrapping at 32 bits:

$$a_k = a_{k-1} + \delta_1, \quad b_k = b_{k-1} + \delta_2 \pmod{2^{32}} \quad (2)$$

The states are then mixed and normalized into a uniform output in $[0,1)$:

$$r_k = \frac{t \oplus (t \gg 14)}{2^{32}}, \text{ where } t = (a_k \oplus (a_k \gg 15)) \cdot (b_k | 1) \quad (3)$$

Separate generator instances, seeded with distinct suffixes appended to the seed string, are used for canvas and audio noise, isolating them from the profile-selection sequence.

D. Value Pools

All spoofed values are drawn from pools of commonly seen profiles, selected based on available statistics provided by sites such as StatCounter [11], the Steam Hardware Survey [12], and the AmIUnique fingerprint dataset [13]:

- **UA profiles:** six Chrome versions on Windows 10 x64 (Chrome 137 through 142). Each entry groups the full UA string, navigator.platform, and the complete userAgentData structure including the shuffled brands array.
- **WebGL profiles:** seven GPU configurations drawn from the Steam Hardware Survey top entries: NVIDIA GeForce RTX 3060, RTX 4060 Laptop GPU, RTX 4060, RTX 3050, GTX 1650, Intel Iris Xe Graphics, and AMD Radeon RX 6600.
- **Memory:** [8, 16, 16, 16, 32, 32, 32] GB, weighted toward 16 GB (41%) and 32 GB (37%), reflecting the shift away from low-memory configurations in current hardware.
- **CPU cores:** [4, 6, 6, 6, 8, 8, 8, 10, 16], weighted toward 6 and 8 cores as the dominant configurations at approximately 28% and 27% respectively.

- **Languages:** seven European language lists (English, German, French, Italian, Spanish, Dutch, Polish), each maintaining the correct primary/secondary structure.
- **Timezone:** five Western/Central European zones with correct Internet Assigned Numbers Authority names, UTC offsets and GMT suffix strings.
- **Screen:** four common desktop resolutions (1920x1080, 1536x864, 1366x768, 2560x1440) with realistic taskbar deductions for available height.
- **Canvas fonts:** twelve common Windows system fonts used to vary canvas text rendering.

IV. IMPLEMENTATION

A. Function Masking

All API interception starts with overriding *Function.prototype.toString*. This step is important because fingerprinting scripts first check whether a function has been interfered or not. The check is simple: call *.toString()* on any browser function, and a real, untouched function will return a string containing *[native code]*. A modified function will return its actual JavaScript source code, immediately revealing that it has been replaced.

To prevent this, the extension maintains a Map called *hookedFunctions* that pairs each hooked function with the name it should report. Before any other hook is installed, the extension saves a reference to the original *Function.prototype.toString* and redefines it using *Object.defineProperty*. The new version checks the map every time it is called. If the target function is found in the map, it returns the expected string, which is exactly what the browser would return for a native function. If the function is not in the map, the call is passed through to the original *toString*, so that unrelated functions continue to behave normally.

Each hook is registered through a helper function called *hideFn(fn, name)*. This function does two things: it adds the function to the *hookedFunctions* map so that *toString* returns the right output, and it overwrites the function's name property using *Object.defineProperty* to match the expected native function's name. This second step is necessary because some detection scripts do not call *toString()* at all, they simply read *fn.name*, which simply returns the name of the function as a string, and check whether it matches what they expect for a native function.

Throughout the rest of the extension, every replaced function is passed through *hideFn* immediately after it is defined. This includes modifications on Navigator properties, Date methods, Canvas operations, WebGL parameters, Audio buffers, and the Worker constructor. As a result, no hooked function in the extension is distinguishable from its original version through either *toString()* inspection or name property checks.

B. Navigator

Chrome sets up many Web API properties as lazy accessors that only appear on the prototype when they are first accessed. Lazy accessors mean the API properties are not created if they

are never accessed. Defining them with `Object.defineProperty` before the first request would add them as new own properties rather than modify existing ones, changing the property enumeration order returned by `Object.getOwnPropertyNames` and showing an immediate sign of a modified structure.

The extension instead wraps `window.navigator` in a Proxy over the real navigator object. The proxy intercepts property reads through its `get trap`, a function that runs whenever someone tries to read a property, and returns spoofed values for the targeted properties (`userAgent`, `appVersion`, `platform`, `deviceMemory`, `hardwareConcurrency`, `language`, `languages`, `vendor`, `webdriver`, `userAgentData`). All other reads, along with the `ownKeys`, `getOwnPropertyDescriptor`, and `has traps`, are forwarded to the real navigator object. This way, `Navigator.prototype` stays completely untouched, and the native property order and descriptor identities are preserved.

This design has one known vulnerability. If a script accesses a navigator property by extracting its getter function directly through `Object.getOwnPropertyDescriptor` and calling it manually, it bypasses the Proxy and reaches the real, unmodified value. This happens because the extension intentionally forwards descriptor lookups to the real navigator object to avoid detection. In practice, this is a difficult attack to trigger, it requires working around the normal way of reading properties rather than simply accessing `navigator.userAgent`. For this reason, it is treated as an accepted trade-off, since it cannot be fully solved in pure JavaScript without support from the browser itself.

The `userAgentData` property returns a constructed object with a spoofed `getHighEntropyValues()` method. This method returns promises that resolve to the correct profile values, including architecture (x86), bitness (64), and a shuffled brands array matching the selected Chrome version. Both `getHighEntropyValues` and `toJSON` are registered as being native through `hideFn`.

C. Canvas

Image-based fingerprinting relies on the fact that subtle differences in how graphics hardware and drivers render the same drawing instructions produce slightly different pixel values, which can be hashed into an almost unique identifier. The extension intercepts the function used to read back pixel data from a drawing surface and perturbs a small number of pixel values by an imperceptible amount before returning them. Because this perturbation is drawn from the same deterministic sequence described in the Design section (III), the same drawing operation performed twice within a session remains exactly the same perturbed result, while the same operation performed in a different session returns a different one. A lookup structure tracks which surfaces have already had this perturbation applied, ensuring that a second read of the same surface returns the already modified values rather than applying a fresh layer of noise on top, an unnecessary and time-consuming action.

A related but distinct vector is the identification of the underlying graphics hardware itself, which scripts can query directly through a diagnostic extension to the graphics API. The

function responsible for answering this query is replaced with one that, for the small number of parameters identifying the hardware vendor and model, returns a value selected from the session's chosen graphics-hardware identity rather than the device's real identity, while leaving every other parameter untouched so that the surrounding rendering behaviour continues to function normally.

D. Audio

A similar technique exists for audio: rendering a short sound through an offline audio-processing pipeline and reading back the resulting numeric waveform produces values that vary slightly across devices due to differences in audio hardware and processing precision, making this output usable as a fingerprint in much the same way as canvas output. The extension intercepts the function that exposes this rendered waveform and adds noise to a limited number of its values. Rather than modifying every sample in the buffer, the extension selects a random window covering 70%-90% of the buffer length. This protects against both a fingerprinting script that simply hashes the entire output and one that performs statistical analysis across the full buffer expecting an untouched signal, while keeping the perturbation small enough to be inaudible.

Fig. 1 and *Fig. 2* show the original and modified audio waveforms, respectively. The two are visually identical, confirming that the injected noise does not affect the perceived audio output. *Fig. 3* shows the difference between the two waveforms. The noise amplitude stays within the range of $\pm 2.0 \times 10^{-5}$. This level of perturbation is far below the human hearing threshold, yet it is sufficient to change the audio fingerprint hash, since fingerprinting algorithms rely on exact sample-level comparisons. The audio noise is generated using a dedicated PRNG instance seeded from the session key.

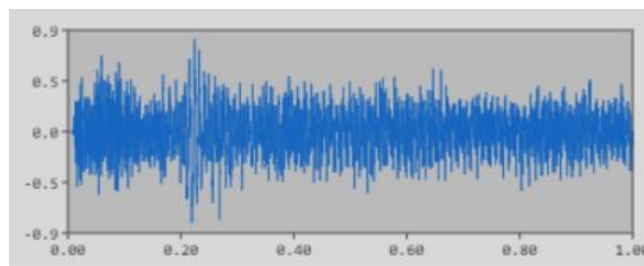


Fig. 1. Original waveform

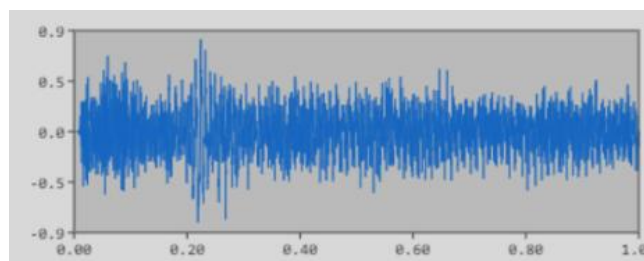


Fig. 2. Modified waveform

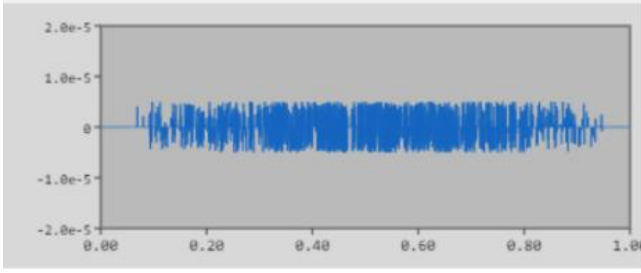


Fig. 3. Difference between the two waveforms

E. Timezone

The extension intercepts all *Date* and *Intl* APIs that expose timezone information. These APIs are connected to each other, so modifying only one creates mismatches that a fingerprinting script can detect by cross-referencing values from different sources.

The date formatting constructor is wrapped so that whenever a new date is created without an explicit timezone option, the extension silently injects the spoofed Internet Assigned Numbers Authority (IANA) zone name. The method that reports the resolved formatting options is also replaced to always return the spoofed zone. The substitute constructor preserves instance identity, so the returned object still passes type checks.

The method that returns the timezone offset returns the value from the selected profile. All getter methods for hours, minutes, seconds, date, day, month, and year apply a UTC-to-local shift using the fixed offset before returning a value.

The string representation methods for dates are implemented manually using modified getters, ensuring that the hour value matches the offset. This avoids using the date formatting API with a timezone name, which would apply real daylight saving time rules and produce a one-hour difference during winter months for zones that observe seasonal time shifts. This decision was made after a real test failure: the main thread correctly applied DST and gave UTC+1 for Berlin in January, while a background worker used the fixed offset and gave UTC+2, producing a visible mismatch between the two contexts.

F. WebGL

A diagnostic extension to the graphics API exposes two parameters that reveal the GPU manufacturer and model. The extension wraps the function responsible for answering these queries on both the WebGL1 and WebGL2 rendering contexts. When the queried parameter matches either of the two hardware identity constants, the replacement returns a vendor or renderer string from the selected GPU profile. All other queries are passed through unchanged.

The same GPU profile is used for both rendering context versions, so a site comparing the two cannot detect a mismatch.

G. WebRTC

WebRTC connection setup can leak the device's local and public IP address. During negotiation, the browser probes external servers to discover these addresses, and the results are available to page scripts without requiring any user permission.

The extension overrides the connection constructor. Regardless of what configuration the page provides, the extension forces the transport policy to be relay-only mode before the connection is created. In this mode, the browser does not attempt to discover its own network address. Instead, all traffic goes through an external relay server, so the user's real IP address is never exposed to the page. The constructor preserves instance identity, so the returned object still passes type checks.

H. Worker Injection

Web Workers run in a separate global context and do not inherit the modifications applied to the main page. A fingerprinting script that creates a Worker and queries browser properties from inside can bypass all the modifications made.

The extension handles this by overriding the Worker constructor. When a page tries to create a Worker, the override does not load the original script directly. Instead, it builds a script that contains two parts: first, a complete copy of all the spoofing modifications adapted for the Worker environment, and second, a call that loads the original worker script. The spoofed values (*user agent*, *timezone*, *GPU identity*, *languages*, etc.) are embedded directly into this script at construction time. This means the Worker sees the same profile as the main thread without needing any message passing between the two.

Inside the Worker, the spoofing modifications are applied to the Worker-specific navigator prototype using property overrides. The timezone, WebGL, and function masking modifications are also replicated.

Workers that use the newer module loading system are not covered by this approach, as the injection technique relies on the older script loading mechanism. This is a known limitation, though module-type Workers are rarely used by fingerprinting scripts in practice.

V. EVALUATION

The extension was evaluated using three methods: a custom multi-user testing framework built with Playwright [10], the CreepJS fingerprinting tests, and the EFF's Cover Your Tracks [8] tool. Together, these methods assess three properties: whether the spoofed identity remains internally consistent within a session, whether it changes across sessions, and whether the user's fingerprint changes in each session.

A. Multi-User Test

We developed an automated testing framework using Playwright that simulates five independent users, each launching a separate Chromium browser instance with the

extension loaded. Each instance uses a fresh user data directory, ensuring a new session seed is generated every time. After loading a minimal test page served by a local HTTP server, the framework collects fingerprint values from every overridden API and runs a set of consistency and diversity checks.

The consistency checks verify that related properties agree with each other within a single session. For *Navigator* properties, the test confirms that the *User-Agent* string and the *User Agent Data* structure report the same Chrome version, that the platform is set to Windows, and that architecture and bitness are correctly reported. For *Screen* and *Window* properties, it verifies that the available dimensions do not exceed the full screen dimensions, that the outer window size matches the reported screen resolution, and that colour depth and pixel ratio are set to expected values. For *Timezone*, the test checks that the offset returned by *getTimezoneOffset* matches the offset embedded in the string returned by *Date.toString()* and *toTimeString()*, and that getter and setter methods for hours, months, and years return correct values. Canvas consistency is tested by reading pixel data twice from the same surface and confirming that both reads produce the same hash. *WebGL* consistency is verified by checking that WebGL1 and WebGL2 contexts report the same GPU identity. Audio consistency is confirmed by reading the same buffer twice and checking that both reads are identical. *WebRTC* is tested by checking that the transport policy is set to relay mode. Worker parity is verified by comparing every fake value read from inside a Worker with the corresponding value from the main thread, including user agent, platform, memory, CPU cores, language, timezone offset, timezone name, date string, and WebGL identity.

All consistency checks passed for all five users across every tested vector.

Group	Checks/user	Total
Navigator	6	30
Screen / Window	6	30
Timezone	8	40
Canvas	3	15
WebGL	5	25
Audio	2	10
WebRTC	1	5
Worker Parity	12	60
toString() Masking	13	65
Total	56	280

Fig. 4. Check groups and pass counts (5 simulated users).

The diversity checks measure how many distinct values appear across the five users for each property. The results show that canvas hash, FingerprintJS visitor ID and FingerprintJS audio hash each produced five distinct values out of five users, resulting in perfect diversity. Timezone zone names produced four distinct values. Screen resolution, memory, CPU cores, and language each produced three distinct values. Chrome version and WebGL GPU produced two distinct values each, which reflect the size of the current value pools (three Chrome

profiles and four GPU profiles). The timezone UTC offset produced only one distinct value across all five users, because all timezone profiles in the current pool are Western or Central European zones that share the same offset during the summer months.

Property	Distinct values	Values observed
Chrome version	2	124, 122, 122, 124, 124
Language	3	It-IT, en-GB, pl-PL, it-IT, en-GB
Screen resolution	3	1366x768, 1440x900, 1440x900, 1440x900, 2560x1440
Memory	3	8, 8, 4, 8, 16 GB
CPU cores	3	8, 8, 4, 12, 12
Timezone zone	4	Warsaw, Rome, Warsaw, Berlin, Paris
Timezone offset	1	-120 for all 5
WebGL GPU	2	AMD RX 580 x4, NVIDIA GTX 1650 x1
Canvas hash	5	unique per session
Audio samples	5	unique per session

Fig. 5. Cross-session diversity across 5 simulated users.

The framework also integrates FingerprintJS, a commercial fingerprinting library, to verify that the extension produces a different visitor ID for each session. All five users received distinct visitor IDs, confirming that the extension successfully prevents cross-session tracking by a widely used production fingerprinting service.

B. CreepJS

CreepJS [6] is an open-source fingerprinting analyzer designed to detect spoofing and inconsistencies. It performs deep checks across multiple browser properties and flags values it considers suspicious or fake.

When tested with the extension active, CreepJS successfully blocked WebRTC, both host and Session Traversal Utilities for NAT (STUN) connections were reported as blocked, confirming that no local IP address was leaked. The timezone was flagged as spoofed, with CreepJS marking the reported offset and location as fake. This is expected: CreepJS compares the reported timezone against values it can infer from other signals, and since the extension replaces the real timezone with a different one, the mismatch is detected. This detection does not identify the user, it only indicates that a privacy tool is active.

The Screen test showed that CreepJS detected a mismatch between the spoofed screen dimensions and certain CSS media query values that the extension does not currently override. The JavaScript-level screen values were replaced correctly, but the browser's own CSS layout queries still reported the real screen dimensions, since these are controlled by the rendering engine and are not accessible to a content script.

The Window version test reported the fingerprint as "new," meaning CreepJS could not match it to any known browser fingerprint in its database. This is a positive result, the spoofed profile does not correspond to any real browser that CreepJS has seen, meaning it does not produce a recognisable or previously tracked identity.

C. Cover Your Tracks

Cover Your Tracks, maintained by the Electronic Frontier Foundation, measures how unique a browser's fingerprint is and whether it is protected against tracking. When tested with the extension active, the tool reported that the browser's fingerprint was randomised and that it had full protection against web tracking. This confirms that from the perspective of a standard fingerprinting analysis, the extension successfully prevents the browser from being uniquely identified.

D. Summary of Results

The evaluation confirms that the extension achieves its primary goals. First, all values remain internally consistent within a session, no mismatch was detected between related properties in any test. Second, the fingerprint changes across sessions, producing distinct profiles for each simulated user and preventing cross-session linking by both open-source and commercial fingerprinting tools.

The evaluation also reveals areas for improvement. The limited size of some value pools (three Chrome versions, four GPU profiles) means that with only five users, some properties do not achieve full diversity. The timezone offset pool produces no diversity at all during summer months. CSS-level screen queries are not currently covered, which creates a detectable mismatch in tools like CreepJS that cross-reference JavaScript values with CSS media queries.

VI. CONCLUSION

This paper presented a Chromium browser extension that defends against browser fingerprinting by replacing the user's real fingerprint with a coherent fake identity drawn from a pool of realistic profiles. The extension operates on two layers: HTTP header rewriting through the *declarativeNetRequest* API and *JavaScript* API interception through a content script injected before any page code executes. A seeded pseudo-random number generator ensures that all spoofed values remain consistent within a session, while a fresh seed is generated for each new session, preventing cross-session tracking.

The implementation covers eight fingerprinting vectors: Navigator properties, Screen and Window dimensions, Timezone, Canvas, WebGL, Audio, WebRTC, and Web Workers. Each vector required addressing specific technical challenges, from the lazy accessor behaviour of Chrome's Navigator prototype to the DST inconsistency between Intl-based and offset-based timezone representations. A function masking mechanism ensures that all replaced functions are indistinguishable from native browser functions through both

toString() inspection and name property checks. Cross-frame propagation ensures that same-origin iframes see the same spoofed identity as the main page.

The evaluation demonstrated that the extension produces internally consistent fingerprints that change between sessions, successfully generating distinct visitor IDs in the FingerprintJS commercial library and achieving full protection according to the EFF's Cover Your Tracks tool.

Several limitations remain. The value pools are currently small, which limits the diversity of generated profiles. Font enumeration is not covered, leaving a potential fingerprinting vector open. CSS media queries report the real screen dimensions, creating a detectable mismatch with the spoofed JavaScript values. TLS-level fingerprinting operates at the network layer and cannot be addressed by a browser extension. Service Workers and module-type Web Workers are not covered by the current injection mechanism.

Future work could address these gaps by expanding the value pools with telemetry data from real-world browser populations, adding CSS-level overrides for screen dimensions, covering font enumeration, and exploring integration with browser-level APIs that could provide deeper access to properties currently outside the reach of a content script.

REFERENCES

- [1] P. Eckersley, "How unique is your web browser?" in Proc. 10th Int. Conf. Privacy Enhancing Technologies (PETs), 2010, pp. 1–18.
- [2] S. Englehardt and A. Narayanan, "Online tracking: A 1-million-site measurement and analysis," in Proc. ACM Conf. Computer and Communications Security (CCS), 2016, pp. 1388–1401.
- [3] Pierre Laperdrix, Benoit Baudry, Vikas Mishra. FPRandom: Randomizing core browser objects to break advanced device fingerprinting techniques. ESSoS 2017- 9th International Symposium on Engineering Secure Software and Systems, Jul 2017, Bonn, Germany. pp.17.
- [4] The Tor Project, "Tor Browser Design Document," 2023. [Online]. Available: <https://2019.www.torproject.org/projects/torbrowser/design/>
- [5] M. Gomez-Boix, P. Laperdrix, and B. Baudry, "Hiding in the crowd: An analysis of the effectiveness of browser fingerprinting at large scale," in Proc. WWW, 2018, pp. 309–318.
- [6] A. Juliot, "CreepJS - Creep with JavaScript," GitHub, 2021. [Online]. Available: <https://abrahamjuliot.github.io/creepjs/>
- [7] Brave Software, "Brave Browser Privacy Features," 2023. [Online]. Available: <https://brave.com/privacy-features/>
- [8] Electronic Frontier Foundation, "Cover Your Tracks," 2020. [Online]. Available: <https://coveryourtracks.eff.org/>
- [9] FingerprintJS Inc., "FingerprintJS - Browser Fingerprinting Library," GitHub, 2024. [Online]. Available: <https://github.com/fingerprintjs/fingerprintjs>
- [10] Microsoft, "Playwright — End-to-End Testing Framework," 2024. [Online]. Available: <https://playwright.dev/>
- [11] StatCounter, "Global Stats — Browser & Screen Resolution Market Share," Available: <https://gs.statcounter.com>
- [12] Valve Corporation, "Steam Hardware & Software Survey," Available: <https://store.steampowered.com/hwsurvey>
- [13] AmlUnique, "Learn how identifiable you are on the Internet," Available: <https://amiunique.org>
